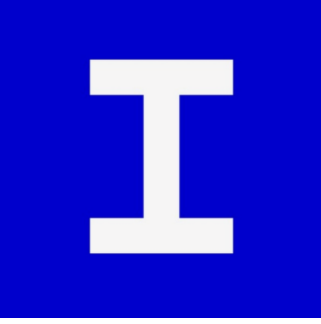


Imperial-X



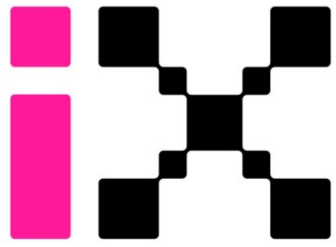
Machine Learning Basics (SPC community)



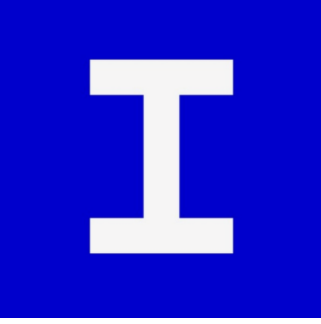
Code and slides

URL:

https://github.com/aidancrilly/ML_Lecture_Demos



Imperial-X



Machine Learning Basics (SPC community)

Dr Aidan Crilly

Imperial College Research Fellow

Former, Schmidt AI in Science Postdoctoral Fellow

What to expect?

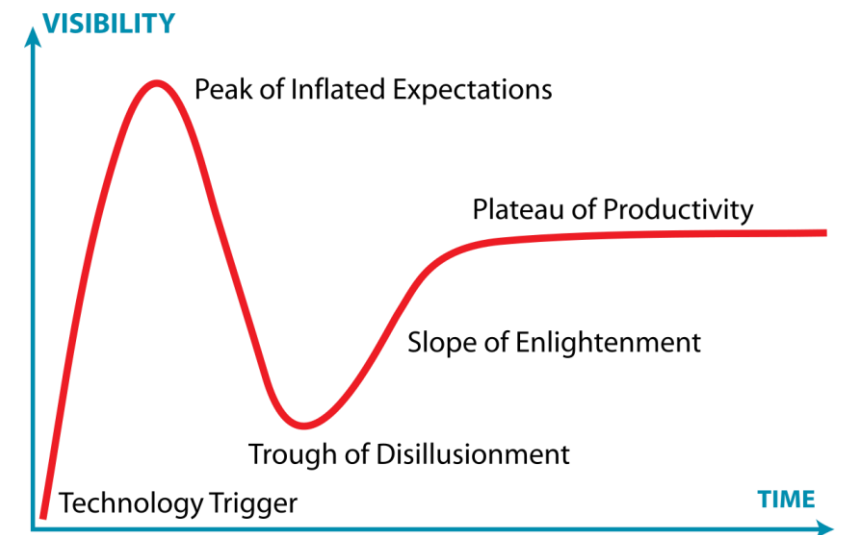
- This talk aims to:
 - Introduce key machine learning (ML) concepts and language
 - Relate ML techniques to more familiar numerical methods
 - Introduce ML techniques which have been used in SPC research
 - Hopefully, be a jumping off point to learning more about ML useful to your own research
- This talk does not aim to:
 - Discuss ML research, we will discuss ML *in* research
 - Be an authority on ML, it would be great if this could become a discussion on use cases

Overview

- What is Machine Learning?
- Regression & Classification
- Parametric Regressors (Non-neural)
 - Ordinary and Non-Linear Least Squares
- Bayesian Inference
- Clustering
- Non-Parametric Regressors
 - Gaussian Processes
- Neural Networks

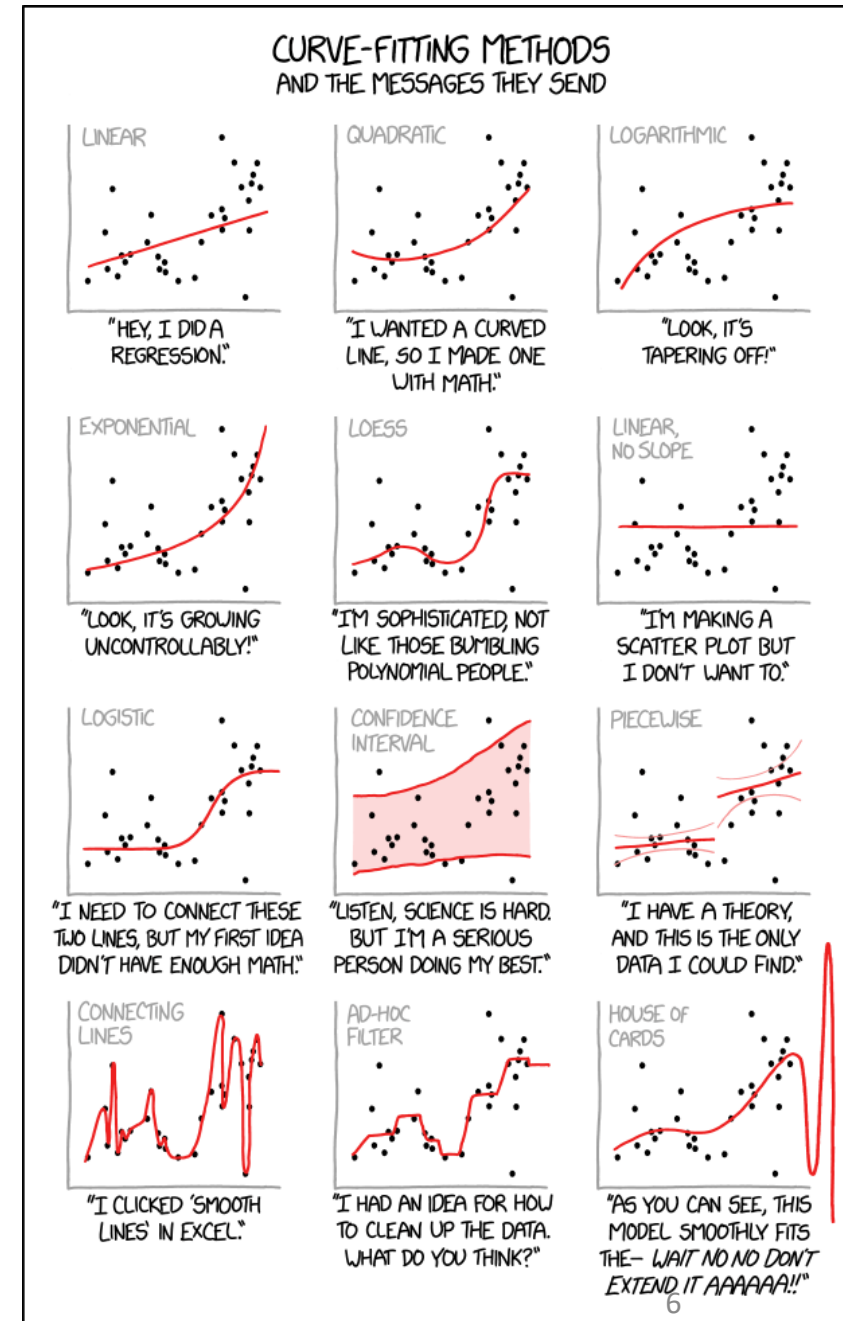
What is “Machine Learning”?

- Numerical modelling which adapts parameters algorithmically to learn the relationship between input and output data
- Parameter evolution often posed as an optimisation problem (nothing new!)
- User often must select ‘hyper-parameters’ which change aspects of the machine learning algorithm



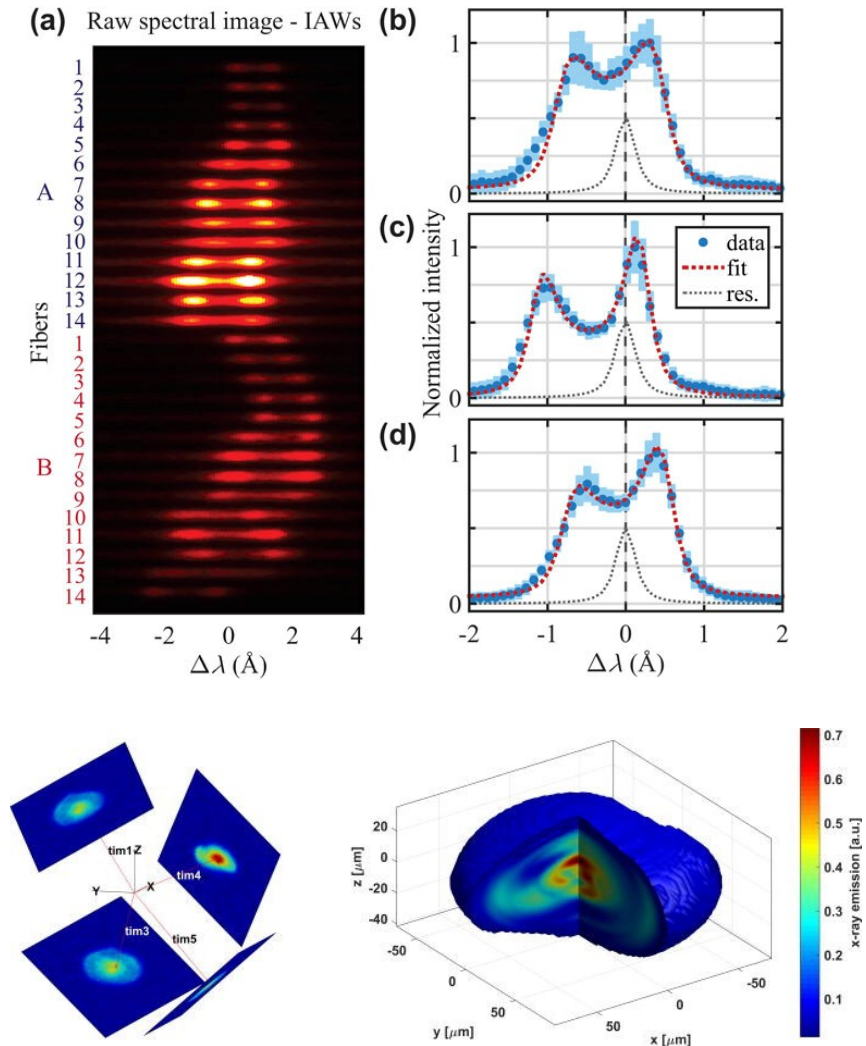
Regression & Classification

- Two of the core ML tasks are regression and classification
- Classification is the task of placing data within a finite number of sets, for example determining if a photo contains a cat or not
- Regression is the task of estimating the functional relationship between a set of input and output variables, for example fitting a straight line to (x,y) data pairs



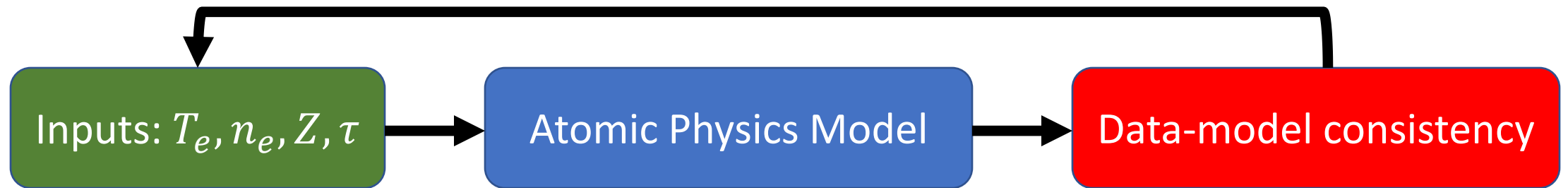
Regression

- Regression problems are ubiquitous in data analysis within the physical sciences
- An entirely non-exhaustive list of examples from my research:
 - Image reconstruction and 3D tomography
 - Thomson scattering spectra
 - Fusion neutron spectroscopy
 - X-ray line spectroscopy
- Typically, reduced models with small number of parameters are used to fit each diagnostic signal separately
- Latent parameters of numerical models can also be learned via regression



Parametric Regressors

- The most common form of regression includes a ‘physics-based’ model which maps input physical parameters (e.g. temperature and density) to the expected diagnostic signature
- For example, line spectroscopy



- The ‘optimal’ set of parameters which match the observed data are sought in a forward-fit process

Ordinary Least Squares



- Say we have a linear model with unknown parameters θ_j :

$$f(x, \theta) = \sum_j \theta_j f_j(x) = \underline{\mathbf{A}} \cdot \boldsymbol{\theta}$$

- And we wish to minimize the weighted least squares distance to the data y_i (we shall see why later):

$$\min_{\boldsymbol{\theta}} \sum_i w_i [y_i - f(x_i, \boldsymbol{\theta})]^2$$

- Defining:

$\mathbf{y}$ = vector of y s

$\boldsymbol{\theta}$ = vector of θ s

$\underline{\mathbf{W}}$ = diagonal matrix of w s

$\underline{\mathbf{A}}$ = matrix of basis function values $f(x_s)$

Ordinary Least Squares



- Say we have a linear model with unknown parameters θ_j :

$$f(x, \theta) = \sum_j \theta_j f_j(x) = \underline{\mathbf{A}} \cdot \underline{\boldsymbol{\theta}}$$

- And we wish to minimize the weighted least squares distance to the data y_i (we shall see why later):

$$\min_{\underline{\boldsymbol{\theta}}} \sum_i w_i [y_i - f(x_i, \underline{\boldsymbol{\theta}})]^2 = \min_{\underline{\boldsymbol{\theta}}} (\underline{\mathbf{y}}^T \underline{\mathbf{W}} \underline{\mathbf{y}} + \underline{\boldsymbol{\theta}}^T \underline{\mathbf{A}}^T \underline{\mathbf{W}} \underline{\mathbf{A}} \underline{\boldsymbol{\theta}} - 2 \underline{\boldsymbol{\theta}}^T \underline{\mathbf{A}}^T \underline{\mathbf{W}} \underline{\mathbf{y}})$$
$$\rightarrow \underline{\mathbf{A}}^T \underline{\mathbf{W}} \underline{\mathbf{A}} \hat{\underline{\boldsymbol{\theta}}} = \underline{\mathbf{A}}^T \underline{\mathbf{W}} \underline{\mathbf{y}}$$

- Therefore, the ‘best fit’ is given by the solution to the ‘normal equations’:

$$\hat{\underline{\boldsymbol{\theta}}} = (\underline{\mathbf{A}}^T \underline{\mathbf{W}} \underline{\mathbf{A}})^{-1} \underline{\mathbf{A}}^T \underline{\mathbf{W}} \underline{\mathbf{y}}$$

For simplicity, assume w is constant going forward...

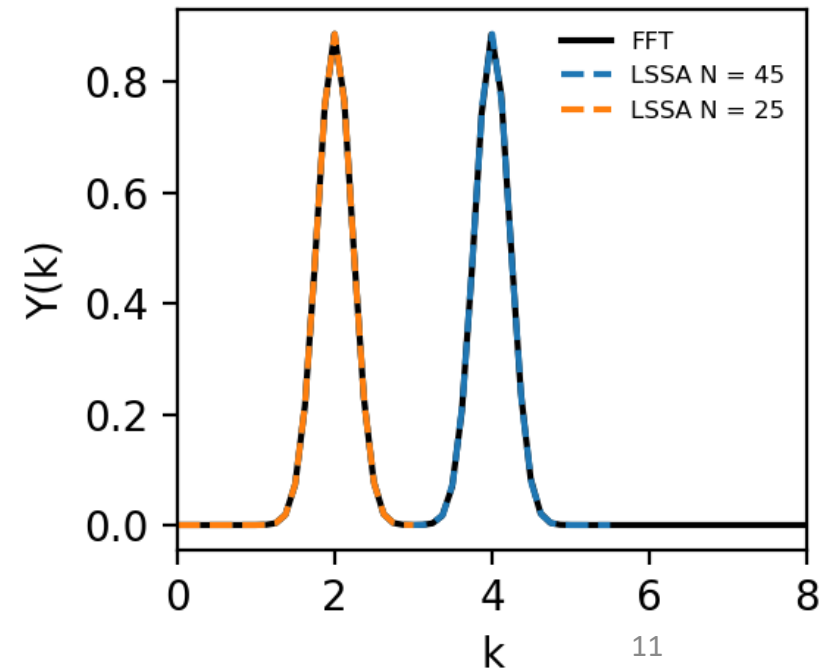
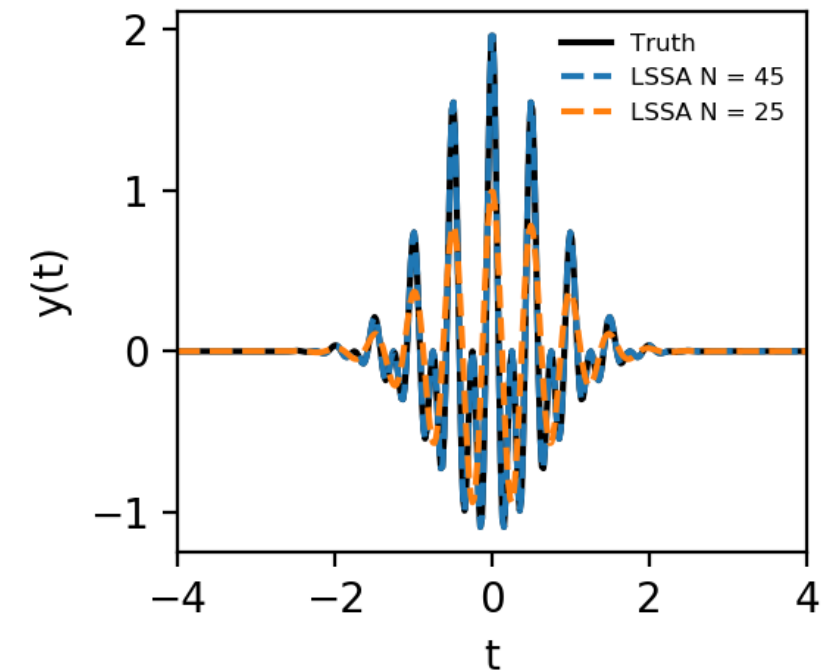
Practical example

- Least-squares spectral analysis (LSSA) is a linear regression problem

$$f(x, \theta) = \sum_{j=0}^N \theta_j \cos \left[\frac{\pi j}{T} t \right] = \underline{\mathbf{A}} \cdot \underline{\boldsymbol{\theta}}$$

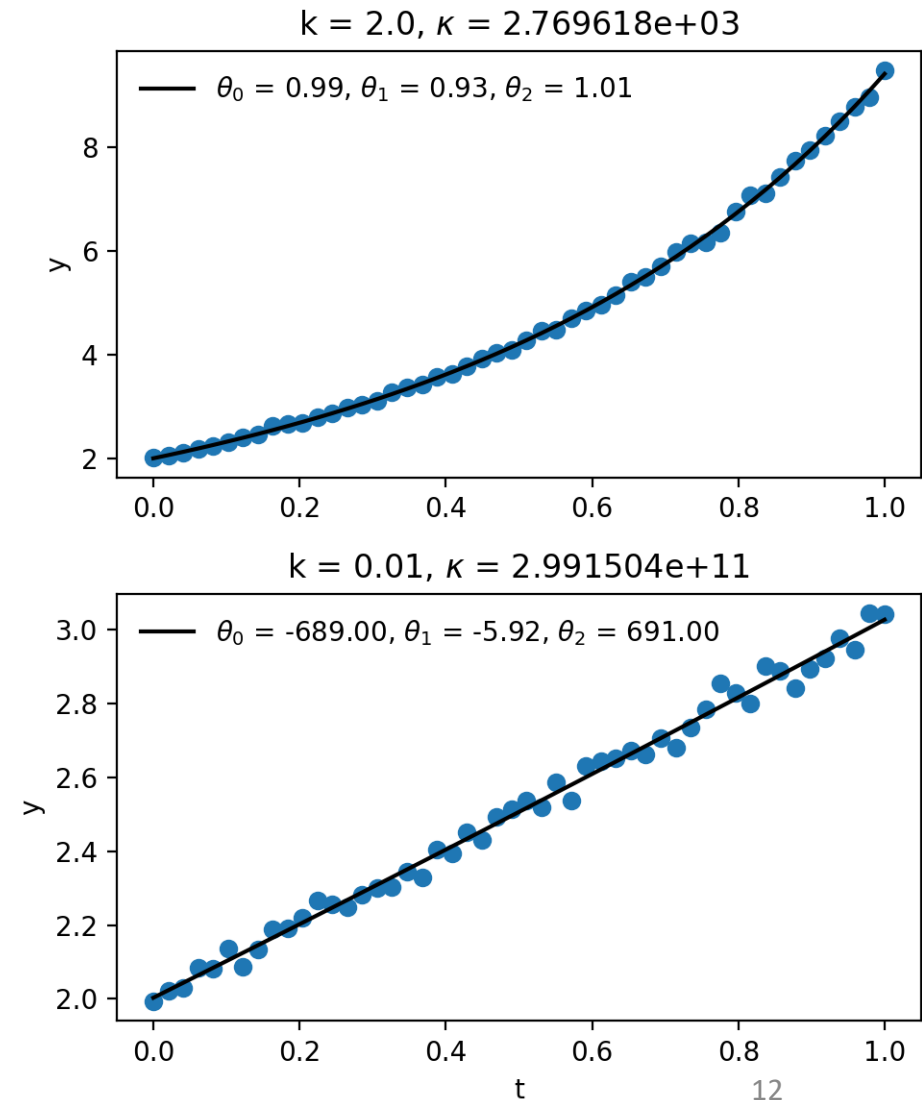
$$\hat{\boldsymbol{\theta}} = (\underline{\mathbf{A}}^T \underline{\mathbf{A}})^{-1} \underline{\mathbf{A}}^T \mathbf{y}$$

- Can handle unequally spaced data, uncertainties and limit the mode number
 - See Lomb-Scargle method
- Not formally a Discrete Fourier Transform



Ill-conditioned or ill-posed problems

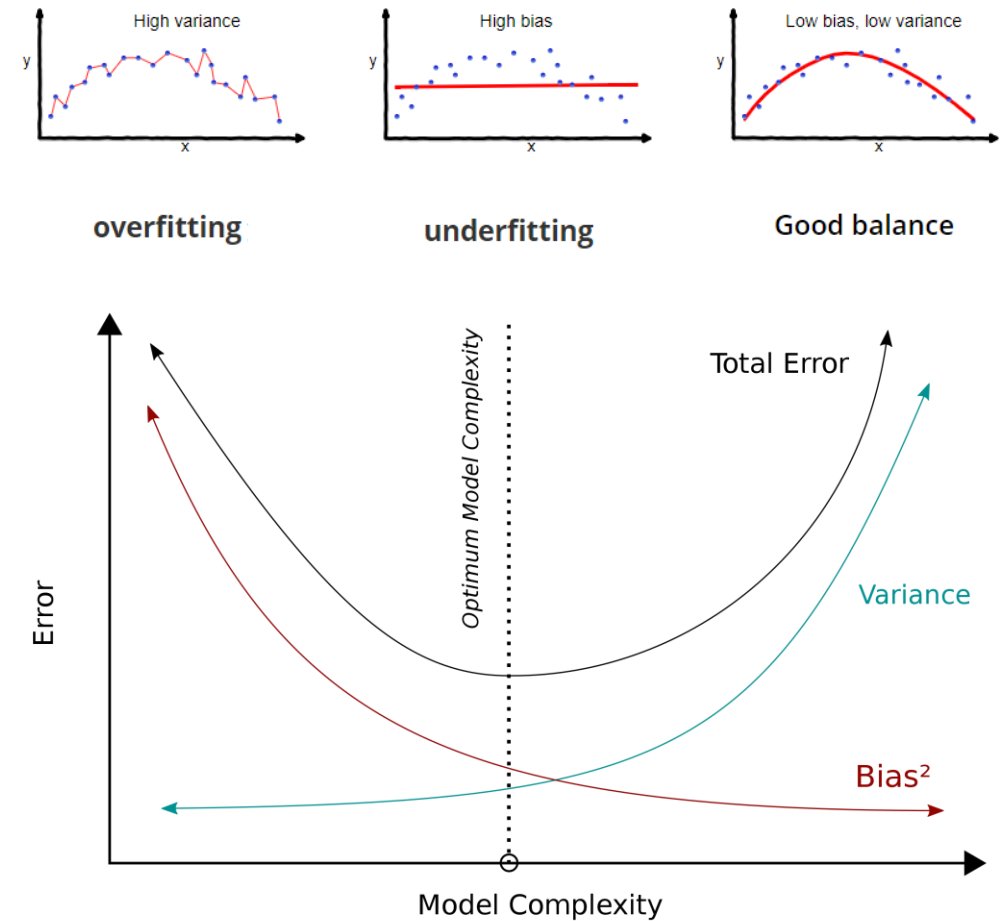
- While linear systems have formal solutions they can be numerical unstable, small errors on one side of the equation can lead to large errors on the other
- One possible cause is degeneracy or collinearity of input parameters
 - e.g. fitting $f(\theta, k, t) = \theta_0 + \theta_1 t + \theta_2 \exp[kt]$, for small kt
- These problems are ‘ill-conditioned’ as quantified by the condition number of the linear system



Regularisation – Bias-variance tradeoff

- Another key ML technique is regularisation
- It aims to reduce variance while introducing bias i.e. prevents overfitting
- Error on *unseen* sample:

$$\text{MSE} = \text{Var}(f) + \text{Bias}(f)^2 + \sigma^2$$



Regularisation

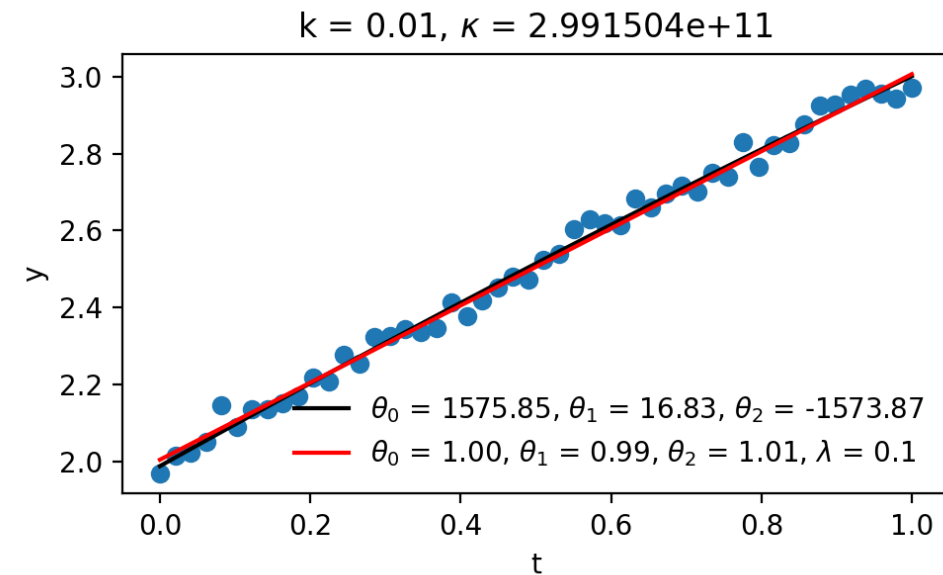
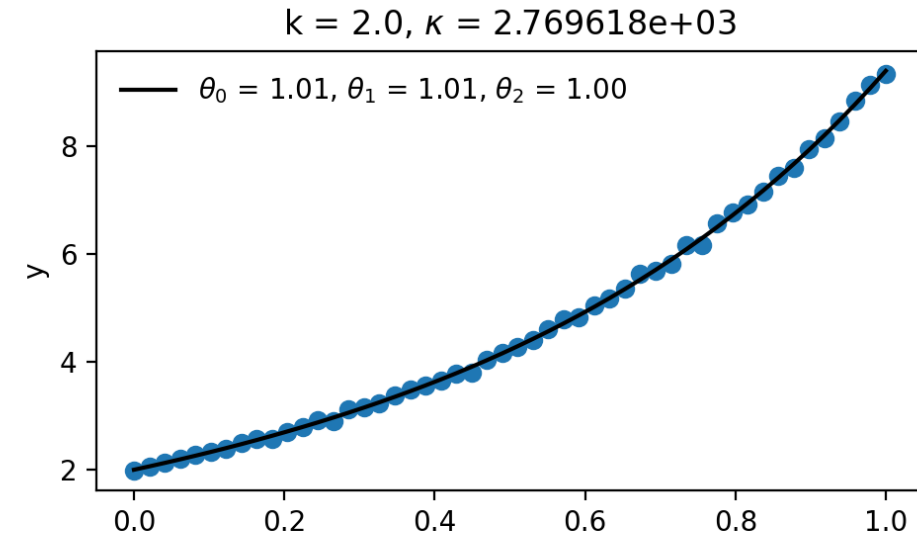
- Another key ML technique is regularisation
- It aims to reduce variance while introducing bias i.e. prevents overfitting

$$\text{MSE} = \text{Var}(f) + \text{Bias}(f)^2 + \sigma^2$$

- Most commonly, Tikhonov regularisation

$$\min_{\theta} \sum_i [y_i - f(x_i, \theta)]^2 + \lambda |\theta|^2$$

- λ is Lagrange multiplier on constraint $|\theta|^2 = 0$
- Other methods include truncated SVD, spectral filtering, parameter priors, and many more



Deconvolution and Regularisation

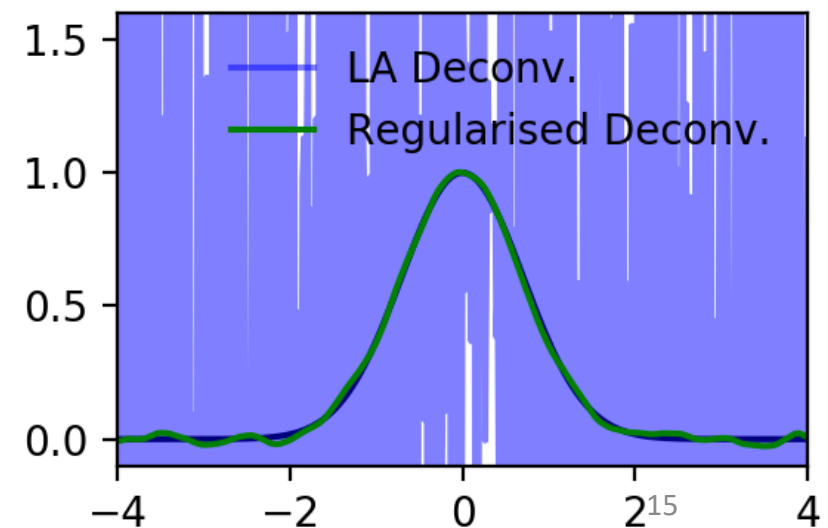
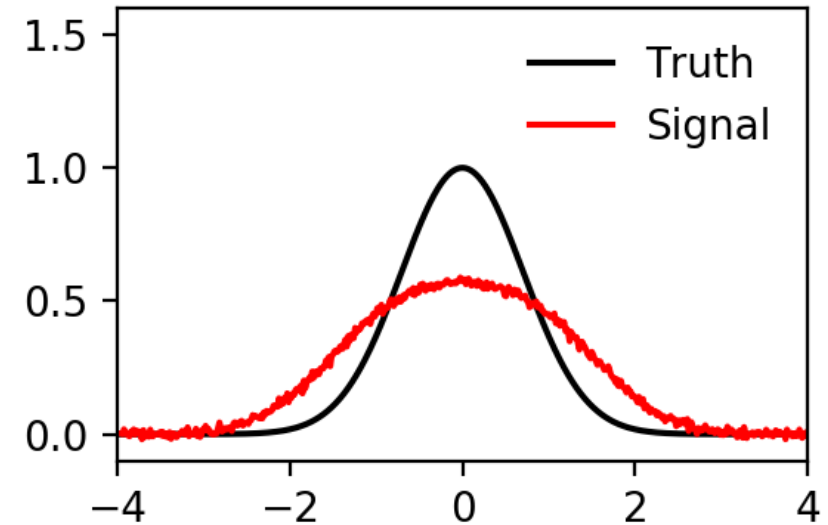
- Convolution is linear

$$F(t) = \int R(\tau - t)f(\tau)d\tau \rightarrow F_i = R_{ij}f_j$$

- However, deconvolution is an ill-posed problem (R_{ij} is *band-limited*)
- Many regularised solution methods for deconvolution
- We will penalise large gradients:

$$\min_{\theta} \sum_i [y_i - R_{ij}f_j]^2 + \lambda (D_{ij}^1 f_j)^2$$

First order finite
difference



Non-Linear Least Squares

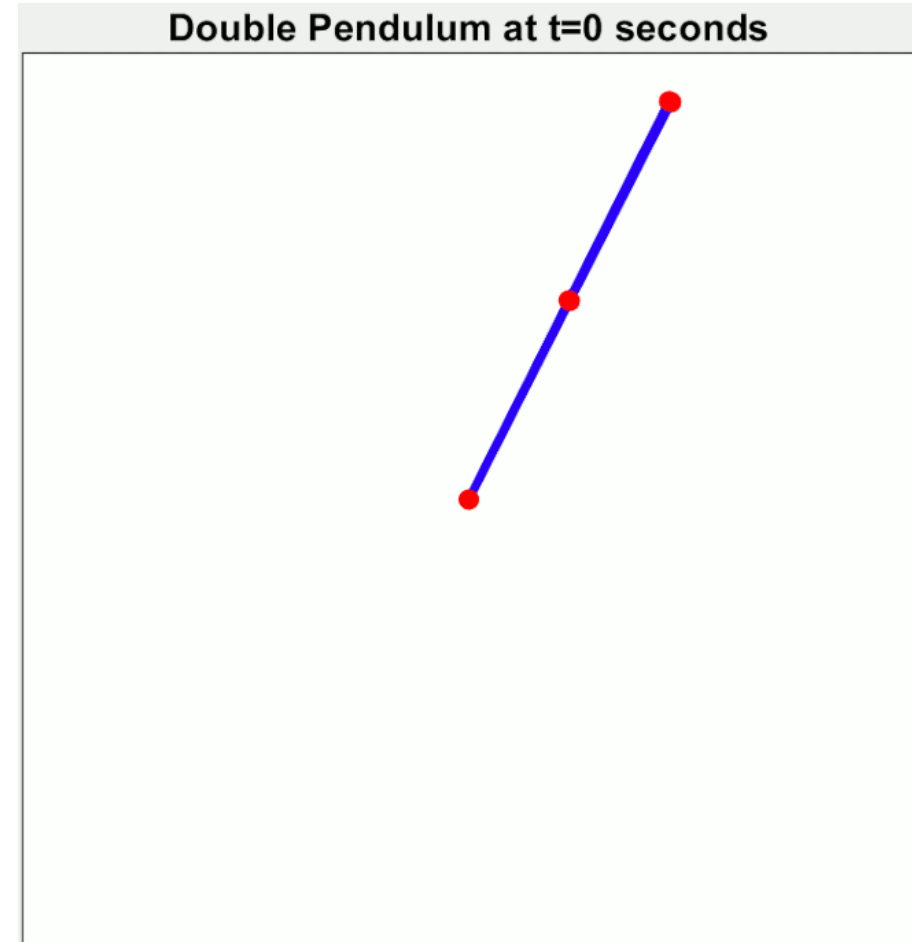
- Say we have a non-linear model with unknown parameters θ_j :

$$f(x, \theta_1 \dots \theta_j \dots \theta_N)$$

- And we are looking for the solution of least squares:

$$\min_{\theta} \sum_i w_i [y_i - f(x_i, \theta)]^2$$

- As you might expect, there is no direct linear algebra solution to this non-linear problem





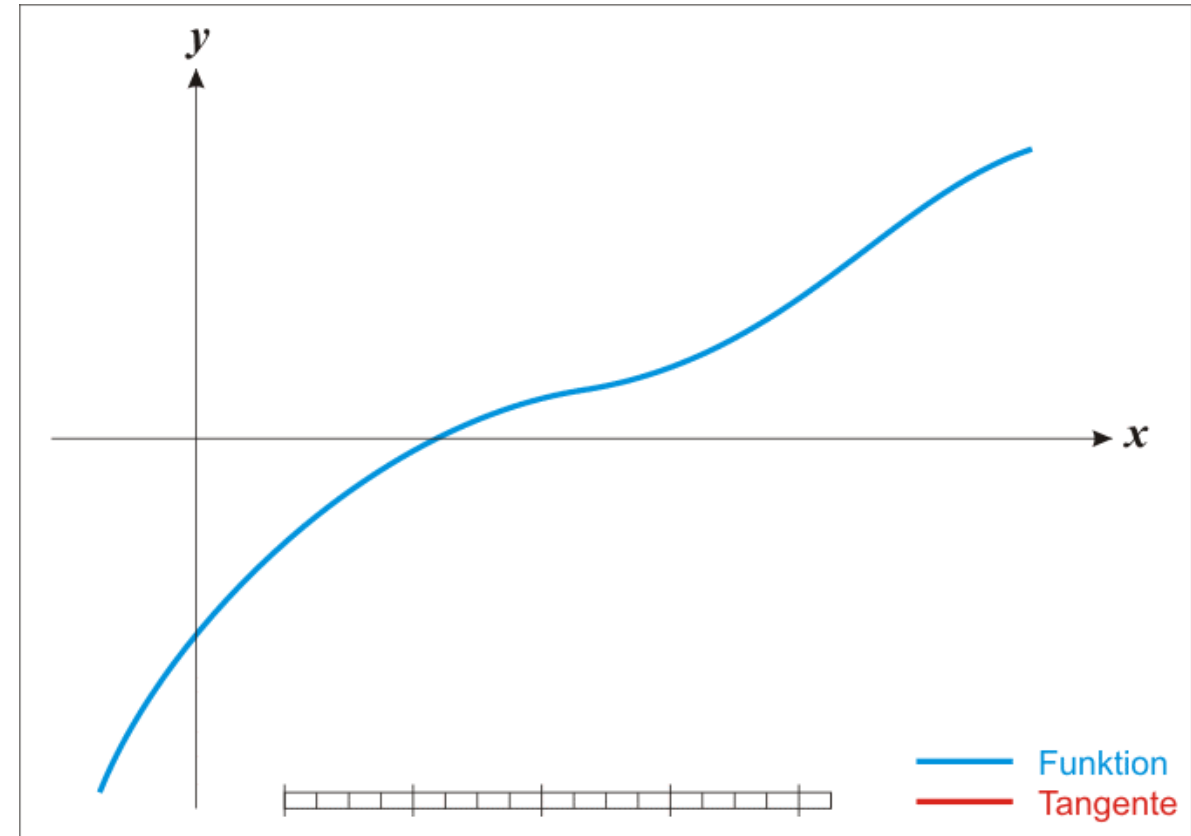
Reminder: Newton-Raphson Method

- Iterative method for finding the root of a function:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$

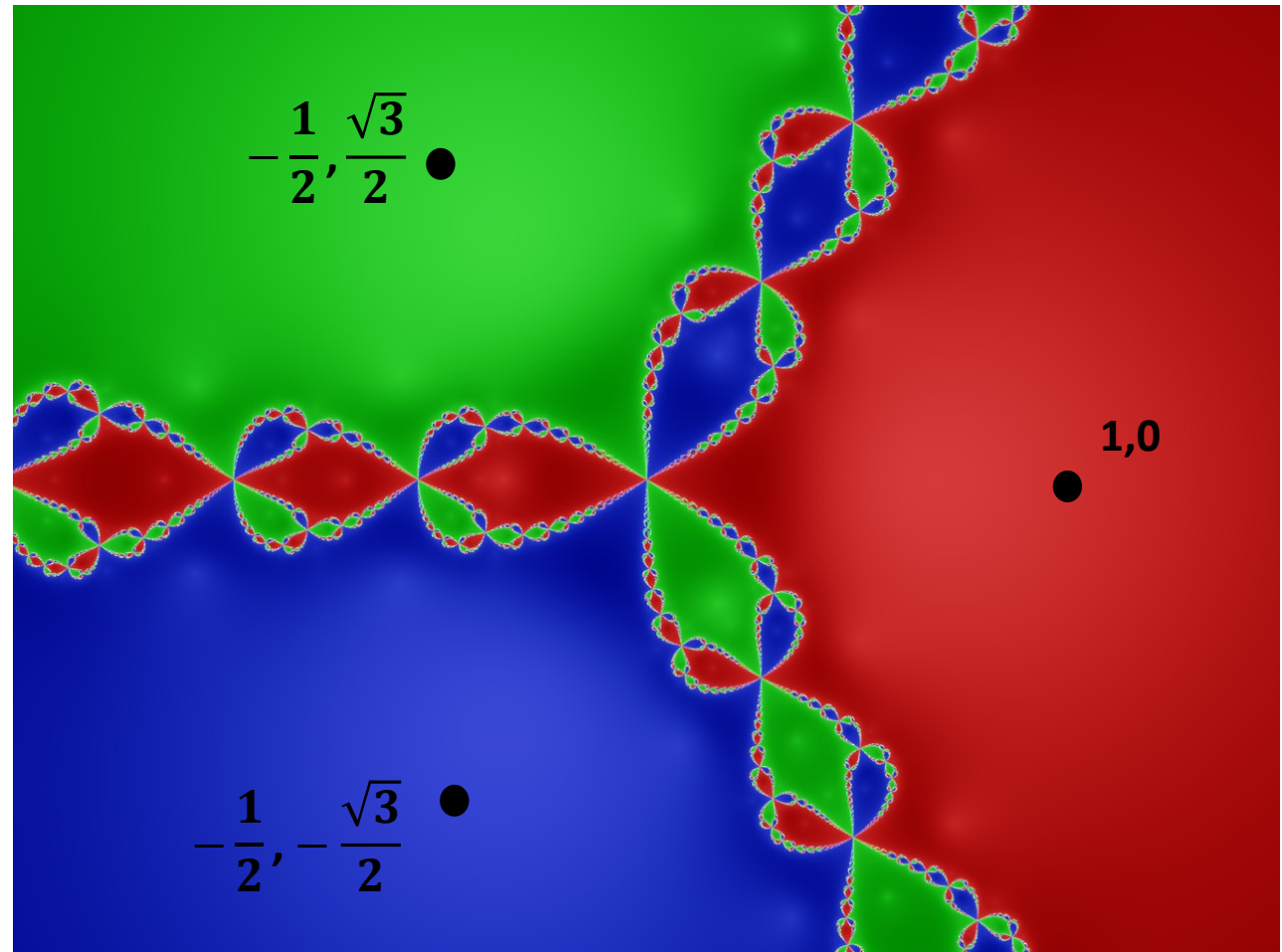
- Minimising a function, $g(x)$, is same as root finding its gradient function, $f(x)$

$$x_{n+1} = x_n - \frac{g'(x_n)}{g''(x_n)}$$



Fun aside: Newton's Fractal

- Depending on starting point, one converges to a different root
- Which root you end up at can have a fractal structure!
- Example: $f(z) = z^3 - 1$
- Example of Holomorphic Dynamics



Non-Linear Least Squares

- We are looking for the solution of least squares:

$$\min_{\boldsymbol{\theta}} S(\boldsymbol{\theta}) = \min_{\boldsymbol{\theta}} \sum_i w_i [y_i - f(x_i, \boldsymbol{\theta})]^2$$

- Lets linearise for small displacements in $\boldsymbol{\theta}$, and drop weights for simplicity:

$$\begin{aligned} & \min_{d\boldsymbol{\theta}} \sum_i [y_i - f(x_i, \boldsymbol{\theta} + d\boldsymbol{\theta})]^2 \\ &= \min_{d\boldsymbol{\theta}} \sum_i \left[y_i - f(x_i, \boldsymbol{\theta}) + \underline{J} \cdot d\boldsymbol{\theta} \right]^2 \end{aligned}$$

- Retrieving a multivariate version of Newton's method:

$$\rightarrow d\boldsymbol{\theta} = \left(\underline{J}^T \underline{J} \right)^{-1} \underline{J}^T (\mathbf{y} - f(\mathbf{x}, \boldsymbol{\theta}))$$

N.B. we have approximated the Hessian as $2\underline{J}^T \underline{J}$, ignoring all 2nd derivatives, so we can get stuck in saddlepoints!

Non-Linear Least Squares

- Retrieving a multivariate version of Newton's method:

$$\rightarrow d\theta = \left(\underline{J}^T \underline{J}\right)^{-1} \underline{J}^T (\mathbf{y} - f(\mathbf{x}, \theta))$$

- What if the Jacobian has large condition number? Regularize

$$\rightarrow d\theta = \left(\underline{J}^T \underline{J} + \lambda \underline{I}\right)^{-1} \underline{J}^T (\mathbf{y} - f(\mathbf{x}, \theta))$$

- If $\underline{J}^T \underline{J}$ dominates, then Newton-Raphson step
- If $\lambda \underline{I}$ dominates, then gradient descent step

$$\rightarrow d\theta = \frac{1}{\lambda} \cdot \underline{J}^T (\mathbf{y} - f(\mathbf{x}, \theta)) = -\frac{1}{2\lambda} \frac{dS(\theta)}{d\theta}$$

N.B. Levenberg-Marquardt algorithm uses the above but adaptively changes λ , this is default algorithm in `scipy.optimize.curve_fit`

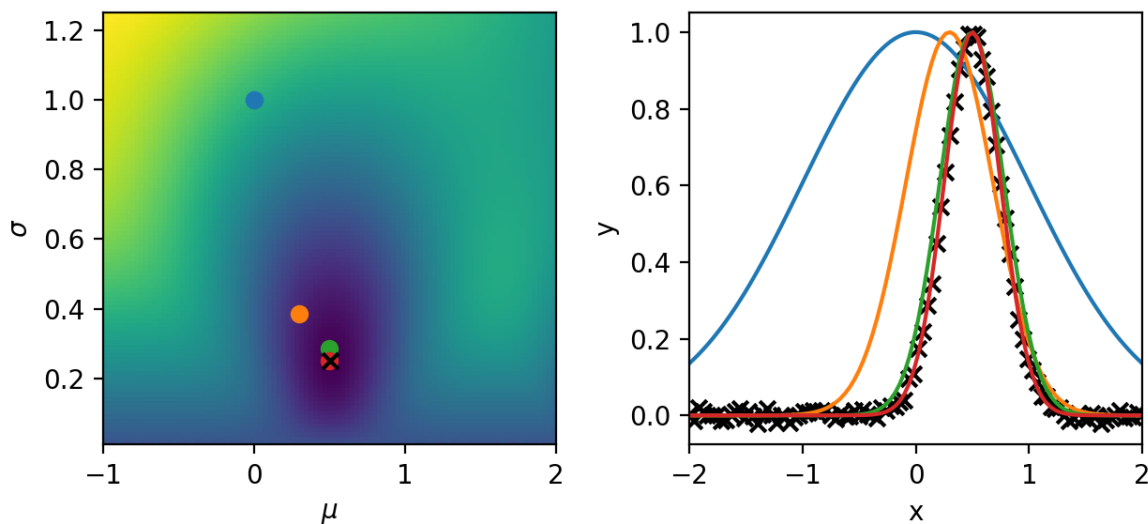
Non-Linear Least Squares

$$\rightarrow d\theta = \left(\underline{J}^T \underline{J} + \lambda \underline{I} \right)^{-1} \underline{J}^T (\mathbf{y} - f(\mathbf{x}, \theta))$$

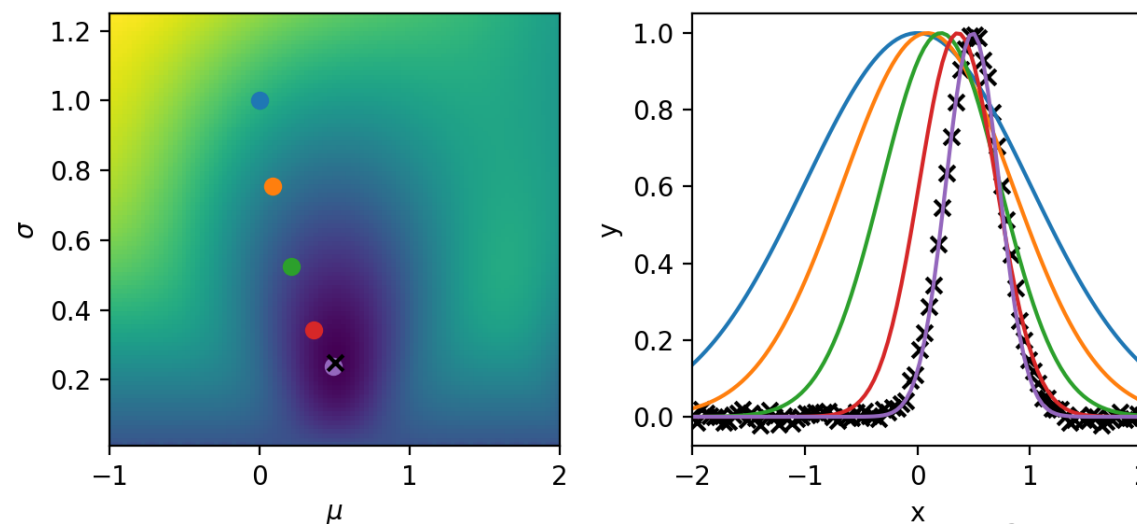
- If $\underline{J}^T \underline{J}$ dominates, then Newton-Raphson step
- If $\lambda \underline{I}$ dominates, then gradient descent step

$$\rightarrow d\theta = \frac{1}{\lambda} \cdot \underline{J}^T (\mathbf{y} - f(\mathbf{x}, \theta)) = -\frac{1}{2\lambda} \frac{dS(\theta)}{d\theta}$$

Newton-Raphson



Gradient Descent



Differentiable Programming

- One might ask, how do you get the Jacobian, $\underline{J}$, to perform gradient descent?
- Numerical derivatives: inaccurate and slow for large number of parameters

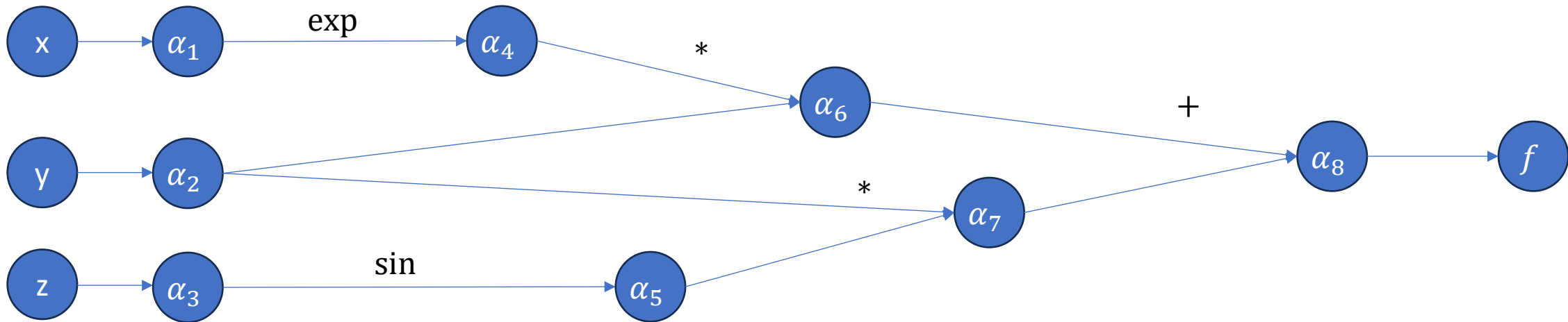
$$\frac{dy}{dx} \approx \frac{y(x + h) - y(x)}{h}$$

- Enter automatic differentiation

Automatic Differentiation Intro

- Every computer program can be written as a computation graph of mathematical 'primitive' functions ($+$, $\times$, $\exp$, $\sin$, etc.)
- For example:

$$f(x,y,z) = y \exp(x) + y \sin(z)$$

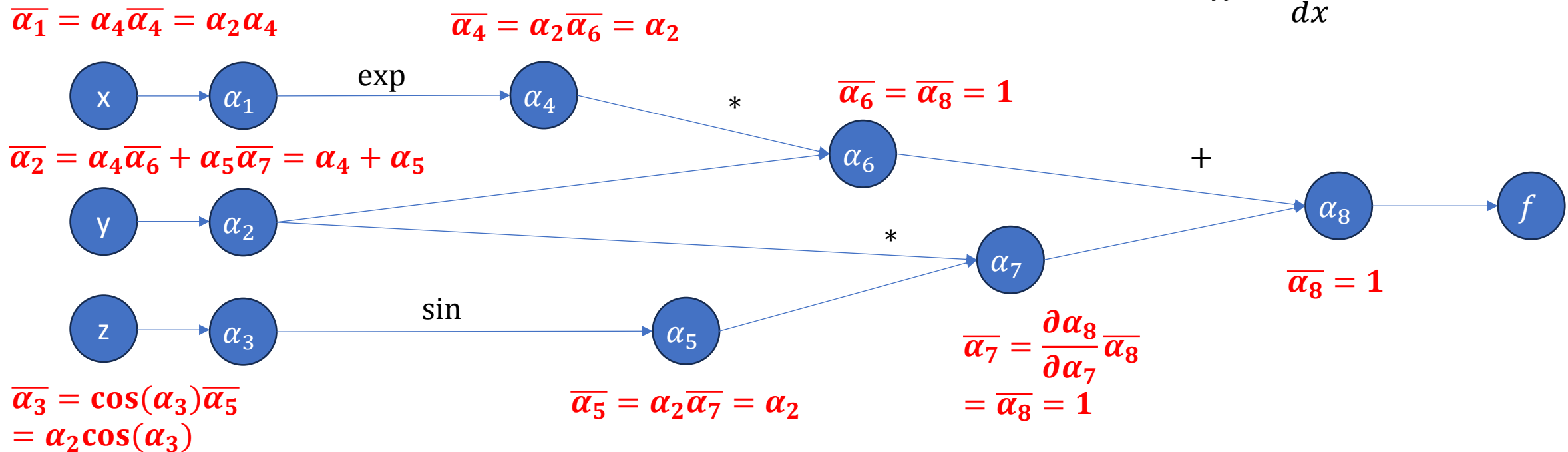


Automatic Differentiation Intro

- In our example:

Use the adjoint:

$$\bar{x} = \frac{df}{dx}$$



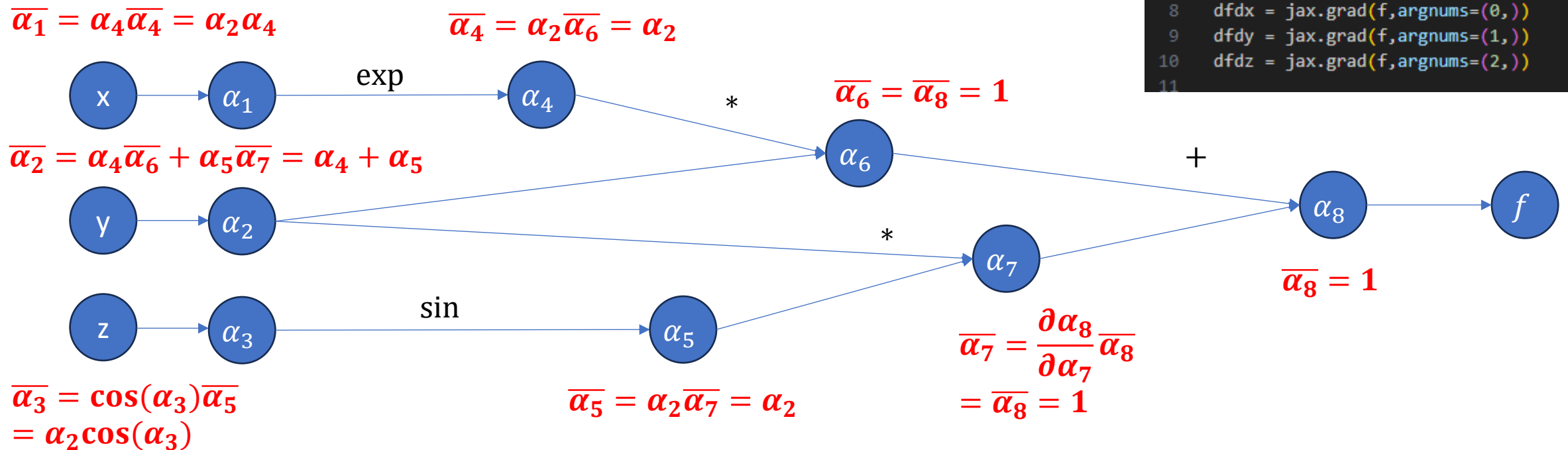
A single 'back-propagation' gives *all* gradients!

Automatic Differentiation Intro

In JAX:

```
1 import jax
2 import jax.numpy as jnp
3 import matplotlib.pyplot as plt
4
5 def f(x,y,z):
6     return y*jnp.exp(x)-y*jnp.sin(z)
7
8 dfdx = jax.grad(f,argnums=(0,))
9 dfdy = jax.grad(f,argnums=(1,))
10 dfdz = jax.grad(f,argnums=(2,))
11
```

- In our example:



A single 'back-propagation' gives *all* gradients!

Automatic Differentiation Intro

- Gradient descent using Automatic Differentiation (AD) gradients:

$$d\theta = \frac{1}{\lambda} \cdot \underline{J}^T(\mathbf{y} - f(\mathbf{x}, \theta)) = -\frac{1}{2\lambda} \frac{dS(\theta)}{d\theta}$$

- Need to:
 1. Define 'loss function' $S(\theta)$ which returns a scalar value given input parameters, θ
 2. Using AD framework to compute $\frac{dS(\theta)}{d\theta}$ using AD
 3. Set an initial value of θ
 4. Identify a suitable learning rate – this can be found by checking that $S(\theta)$ is decreasing after θ is updated by $d\theta$

*Other optimisers can use AD gradients,
we shall cover them later*

Using regression for inference

- Finding the 'best fit' is an optimisation problem
- In inference, we need additional information – the uncertainties

Bayes Theorem and Likelihoods

- Bayes' Theorem states:

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} = \frac{\text{Likelihood x Prior}}{\text{Evidence}}$$

- Canonical example, medical testing:

$$P(\text{ill}|+) = \frac{P(+|\text{ill})P(\text{ill})}{P(+)} = \frac{\text{Sensitivity x Prevalence}}{\text{True Positive} + \text{False Positive}}$$

- Observations always have uncertainty, therefore Bayes rule must be used in parameter estimation



Letter

LII. An essay towards solving a problem in the doctrine of chances. By the late Rev. Mr. Bayes, F. R. S. communicated by Mr. Price, in a letter to John Canton, A. M. F. R. S

Thomas Bayes

Published: 01 January 1763 | <https://doi.org/10.1098/rstl.1763.0053>

Abstract

Dear Sir, I Now send you an essay which I have found among the papers of our deceased friend Mr. Bayes, and which, in my opinion, has great merit, and well deserves to be preserved.

Bayesian Inference

- We wish to find the most likely physical parameters which describe the data:

$$P(\text{model parameters given data}): P(\boldsymbol{\theta}|\mathbf{y}) \propto P(\boldsymbol{\theta})P(\mathbf{y}|\boldsymbol{\theta})$$

- Define a likelihood that *given* the model, $f(\boldsymbol{\theta}, \mathbf{x})$, you would observe the data, say Gaussian errors:

$$P(\boldsymbol{\theta}|\mathbf{y}) \propto P(\boldsymbol{\theta}) \prod_i \exp\left(-\frac{(y_i - f(x_i, \boldsymbol{\theta}))^2}{2\sigma_i^2}\right)$$

Bayesian Inference

- We wish to find the most likely physical parameters which describe the data:

$$P(\text{model parameters given data}): P(\boldsymbol{\theta}|\mathbf{y}) \propto P(\boldsymbol{\theta})P(\mathbf{y}|\boldsymbol{\theta})$$

- Define a likelihood that *given* the model, $f(\boldsymbol{\theta}, \mathbf{x})$, you would observe the data, say Gaussian errors:

$$P(\boldsymbol{\theta}|\mathbf{y}) \propto P(\boldsymbol{\theta}) \prod_i \exp\left(-\frac{(y_i - f(x_i, \boldsymbol{\theta}))^2}{2\sigma_i^2}\right)$$

- Take the logarithm to separate likelihood and priors

$$\propto \log(P(\mathbf{y}|\boldsymbol{\theta})) + \log(P(\boldsymbol{\theta})) = - \sum_i w_i [y_i - f(x_i, \boldsymbol{\theta})]^2 + R(\boldsymbol{\theta})$$

Least squares

Regularisation

Maximal posterior = Minimal least squares



Bayesian Inference – Laplace's Method

- If we expand the log posterior about its optimum:

$$-\log(P(\boldsymbol{\theta}|\mathbf{y})) = F(\boldsymbol{\theta}) \approx F(\boldsymbol{\theta}^*) + \underbrace{J_F(\boldsymbol{\theta})}_{0} (\boldsymbol{\theta} - \boldsymbol{\theta}^*) + \frac{1}{2} H_F(\boldsymbol{\theta}) (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^2$$

- We see the posterior is locally a multi-variate normal:

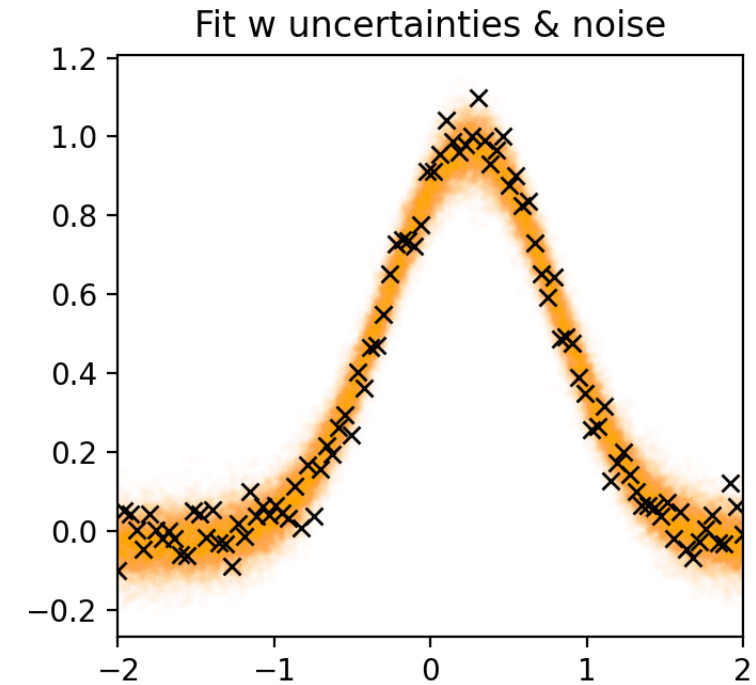
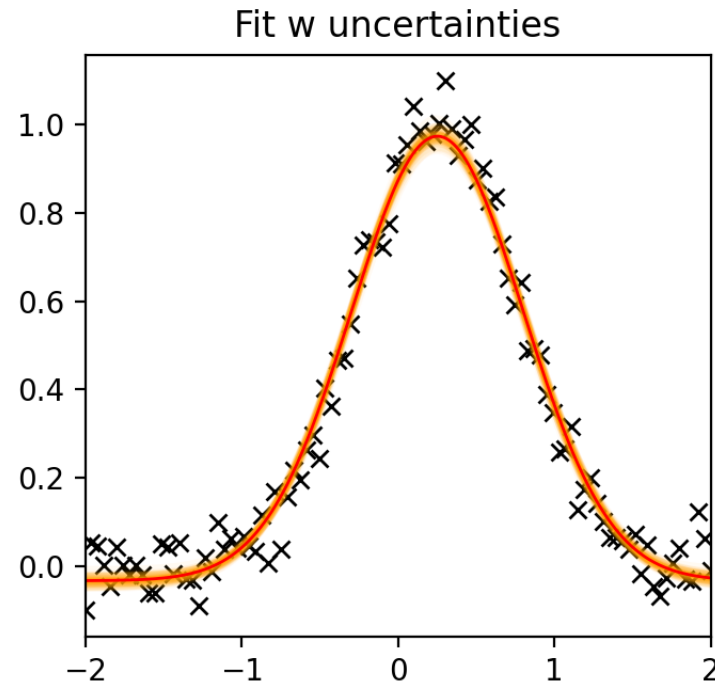
$$P(\boldsymbol{\theta}|\mathbf{y}) \sim \exp \left[-\frac{1}{2} H_F(\boldsymbol{\theta}) (\boldsymbol{\theta} - \boldsymbol{\theta}^*)^2 \right]$$

- The Hessian gives us the covariance matrix – this is what scipy's curve_fit is doing

Example in PythonJAX – Differentiable Programming

- Following our non-linear least squares example, let's fit a Gaussian to data with uncertainties estimated using Laplace's method

```
def neg_log_likelihood(mu,sig,A,B,x_data,y_data,yerr):  
    """ Function to compute the negative log likelihood of our fit to data """  
    y_model = Gaussian(x_data,mu,sig,A,B)  
    return 0.5*jnp.sum(((y_data-y_model)/yerr)**2)  
  
""" Gradient of neg_log_likelihood function w.r.t. input parameters """  
# Here I am doing each gradient explicitly, but it can be done vectorially  
# See how Hessian is done  
gradL_mu = jax.grad(neg_log_likelihood,argnums=0)  
gradL_sig = jax.grad(neg_log_likelihood,argnums=1)  
gradL_A = jax.grad(neg_log_likelihood,argnums=2)  
gradL_B = jax.grad(neg_log_likelihood,argnums=3)  
  
# N.B. we are going to perform gradient descent so  
# the Jacobian of interest is the one with respect to the loss and not the model function  
def Jacobian(mu,sig,A,B,x_data,y_data,yerr):  
    """ Construct the Jacobian using the Automatic Differential gradient functions """  
    J0 = gradL_mu(mu,sig,A,B,x_data,y_data,yerr)  
    J1 = gradL_sig(mu,sig,A,B,x_data,y_data,yerr)  
    J2 = gradL_A(mu,sig,A,B,x_data,y_data,yerr)  
    J3 = gradL_B(mu,sig,A,B,x_data,y_data,yerr)  
    J = jnp.array([J0,J1,J2,J3])  
    return J  
  
def Hessian(mu,sig,A,B,x_data,y_data,yerr):  
    """ Construct the Hessian using the Automatic Differential gradient functions """  
    H = jax.hessian(neg_log_likelihood,argnums=(0,1,2,3))(mu,sig,A,B,x_data,y_data,yerr)  
    return jnp.array(H).reshape(4,4)  
  
# Truth values of inputs
```



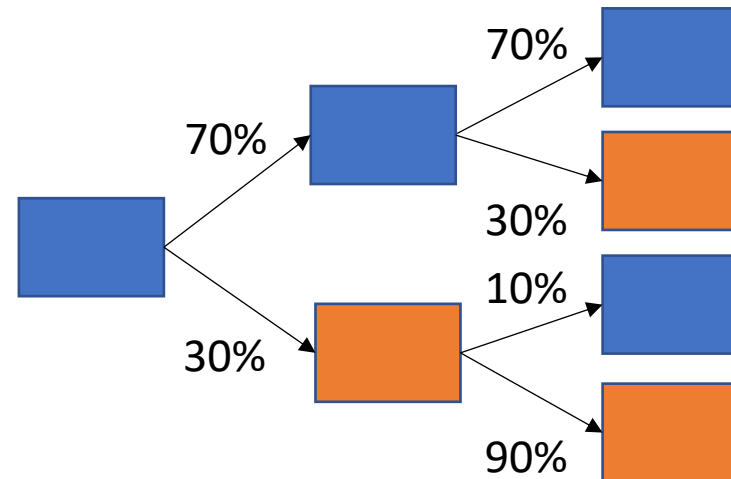
$$\underline{\Sigma} = \underline{H}^{-1}$$

Bayesian Inference: Markov Chain Monte Carlo (MCMC)

- We wish to find the posterior distribution of some parameters but we can only evaluate a function *proportional* to it (Likelihood x Prior)



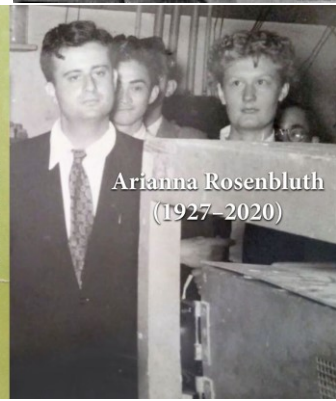
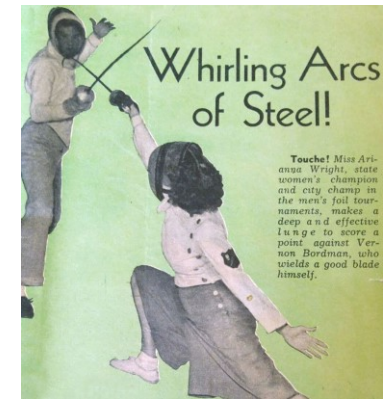
Markov chain:
A sequence of possible events where the next state only depends on the current state

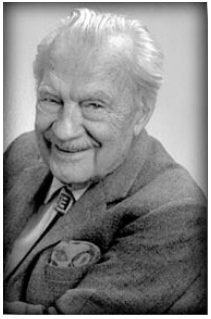


Monte Carlo:
A class of numerical algorithm which involves repeated random sampling



- MCMC: Construct a Markov chain which has an equilibrium distribution of the posterior





Metropolis-Hastings* Algorithm

- $P(x)$ is the probability distribution function we want to sample, $f(x) \propto P(x)$:
1. Starting at x_i , randomly select a point x^* using jumping function $J(x^*|x_i)$
 2. Compute the acceptance ratio, $\alpha = f(x^*)/f(x_i)$ and a random number $u \in [0,1]$
 - a) If $u \leq \alpha$, accept $x^* \rightarrow x_{i+1}$
 - b) Else, reject and $x_i \rightarrow x_{i+1}$

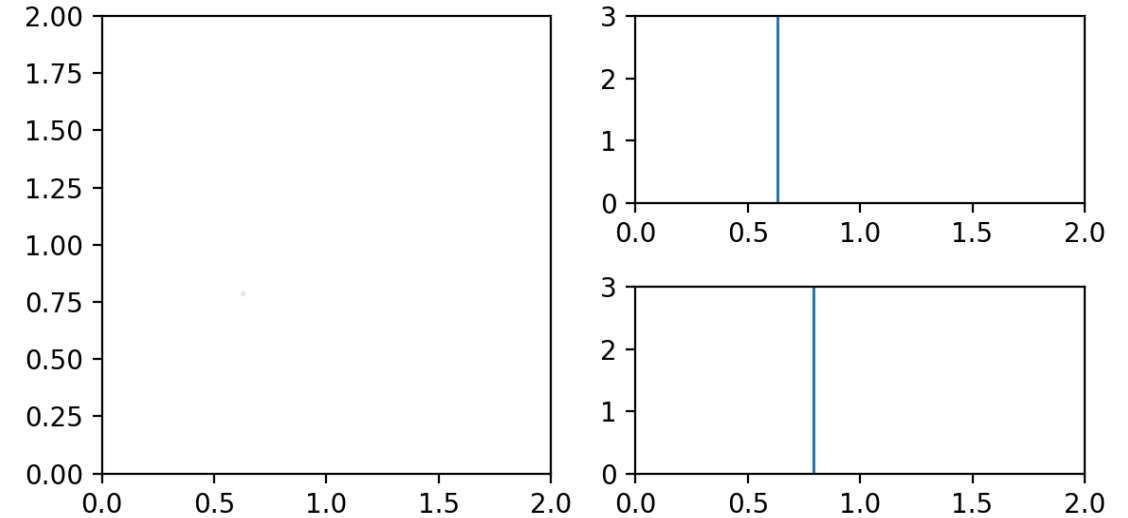
* Arguably, the Rosenbluth algorithm

Equation of State Calculations by Fast Computing Machines

NICHOLAS METROPOLIS, ARIANNA W. ROSENBLUTH, MARSHALL N. ROSENBLUTH, AND AUGUSTA H. TELLER,
Los Alamos Scientific Laboratory, Los Alamos, New Mexico

AND

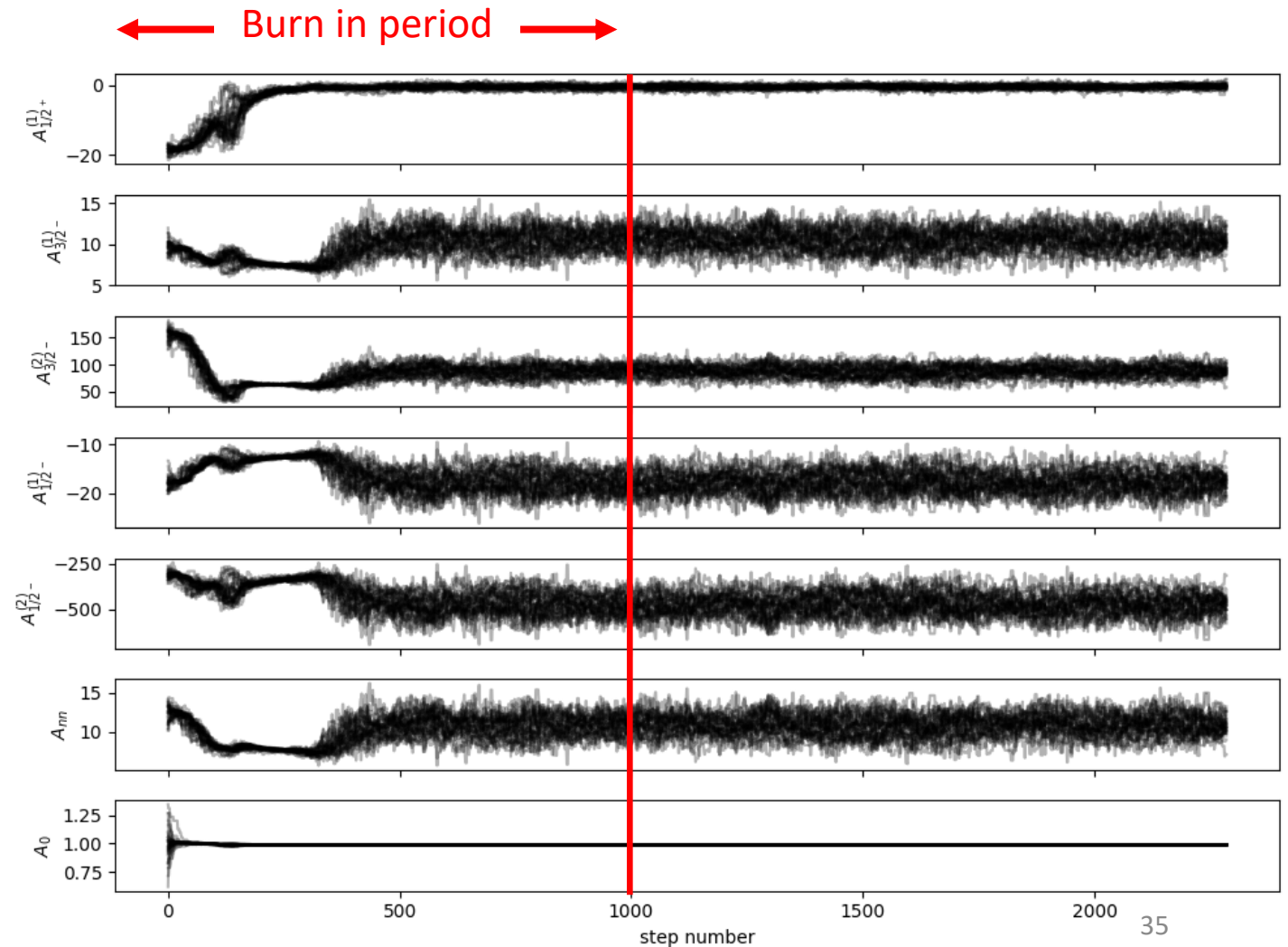
EDWARD TELLER,* *Department of Physics, University of Chicago, Chicago, Illinois*
(Received March 6, 1953)



```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     d = x - mu
6     arg = np.dot(d.T, np.dot(inv_corr, d))
7     return np.exp(-0.5*arg)
8
9 def J(x):
10    y = np.random.multivariate_normal(mean=x, cov=jump_size*np.eye(x.size))
11    return y
12
13 # f parameters, multivariate Gaussian
14 mu = np.array([1.0, 1.0])
15 corr = np.array([[0.05, 0.02], [0.02, 0.05]])
16 inv_corr = np.linalg.inv(corr)
17
18 # Jump function properties
19 jump_size = 0.0025
20
21 # Metropolis algorithm with
22 chain_length = 50000
23 x = 2*np.random.rand(2)
24
25 x1_arr = np.array([])
26 x2_arr = np.array([])
27 u = 1.0
28 alpha = 0.0
29 for i in range(chain_length):
30     while(u > alpha):
31         y = J(x)
32         u = np.random.rand()
33         alpha = f(y)/f(x)
34     x = y
35     u = 1.0
36     alpha = 0.0
37     x1_arr = np.append(x1_arr, x[0])
38     x2_arr = np.append(x2_arr, x[1])
```

MCMC in practice

- Good packages: PyMC, *emcee*
- Define your log probability
- Run # walkers for sampling
- Inspect chains and auto-correlation time
- Chop and thin chain for analysis

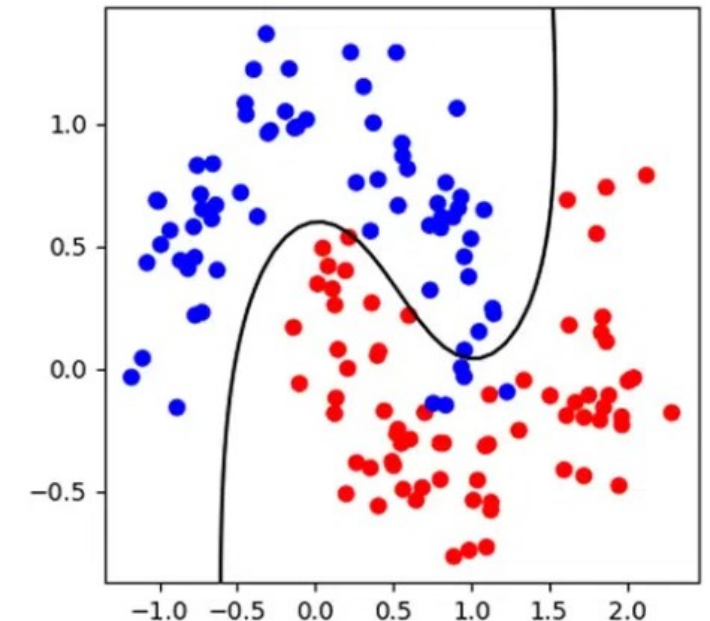
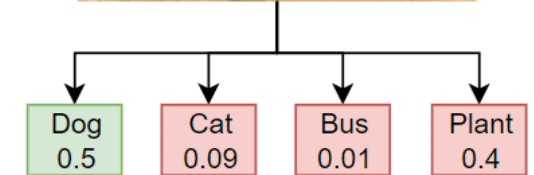


BREAK

Classification - clustering

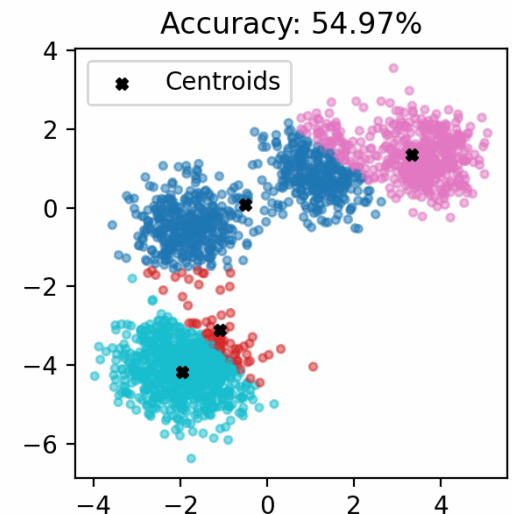
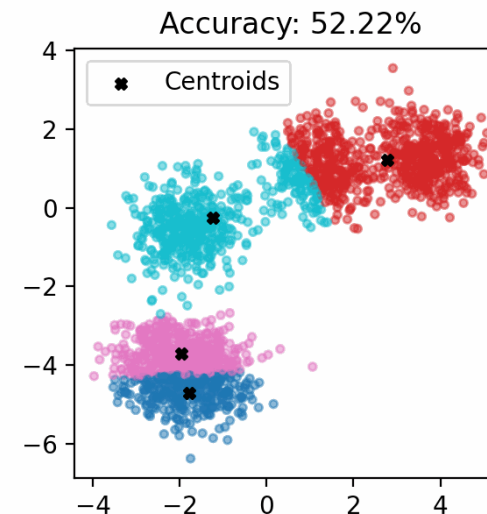
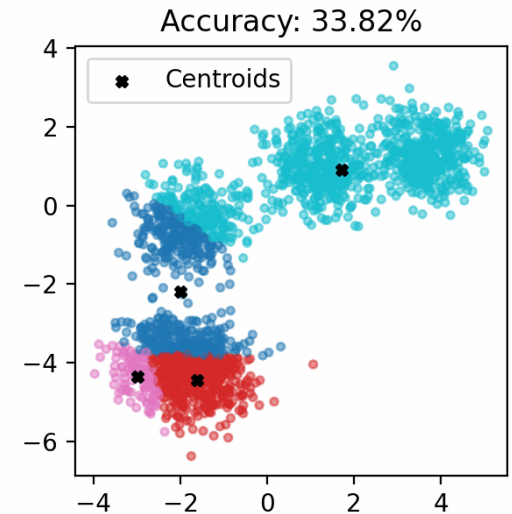
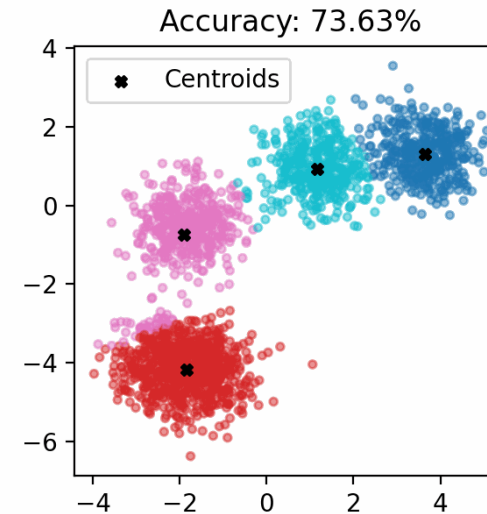
- Clustering is an unsupervised learning task (data has no labels) where data is sorted into a discrete number of sets
- Sorting is performed based on the points' proximity in some space
- Example: K-means clustering
 - Data $\mathbf{x}$ is sorted into k sets based on minimising the least squared distance between points in the set

Multiclass Classification



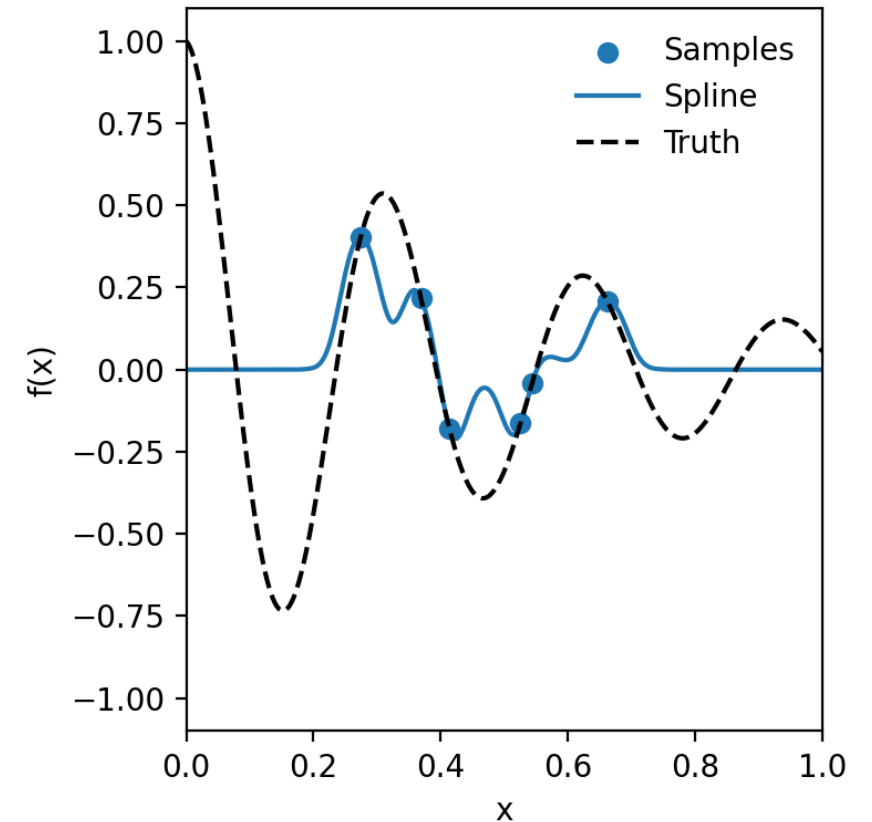
Classification – K-means clustering

- K-means clustering
 - Data x is sorted into k sets based on minimising the least squared distance between points in the set
- Simple algorithm:
 1. Random k centroids are placed within the space
 2. Data is assigned to the centroids based on their distance
 3. Centroids are moved to reflect centroid of assigned data
 4. Iterate to convergence (steps 2-3)



Non-Parametric Regressors

- In the absence of a parametric model, one might want a generic fitting model
- Increasing in various levels of complexity:
 - Nearest-neighbour interpolation
 - Splines
 - Gaussian processes
- Fitting function modifies as you introduce more data? That's a nonparametric regressor



Spline fits are linear least squares:

$$f(x) = \sum_i a_i B(x_i, x)$$

Given data, construct B_{ij} at points x_j

Optimal amplitudes then:

$$\hat{\underline{a}} = (\underline{\underline{B}}^T \underline{\underline{B}})^{-1} \underline{\underline{B}}^T \underline{y}$$

Conditioning multivariate normals

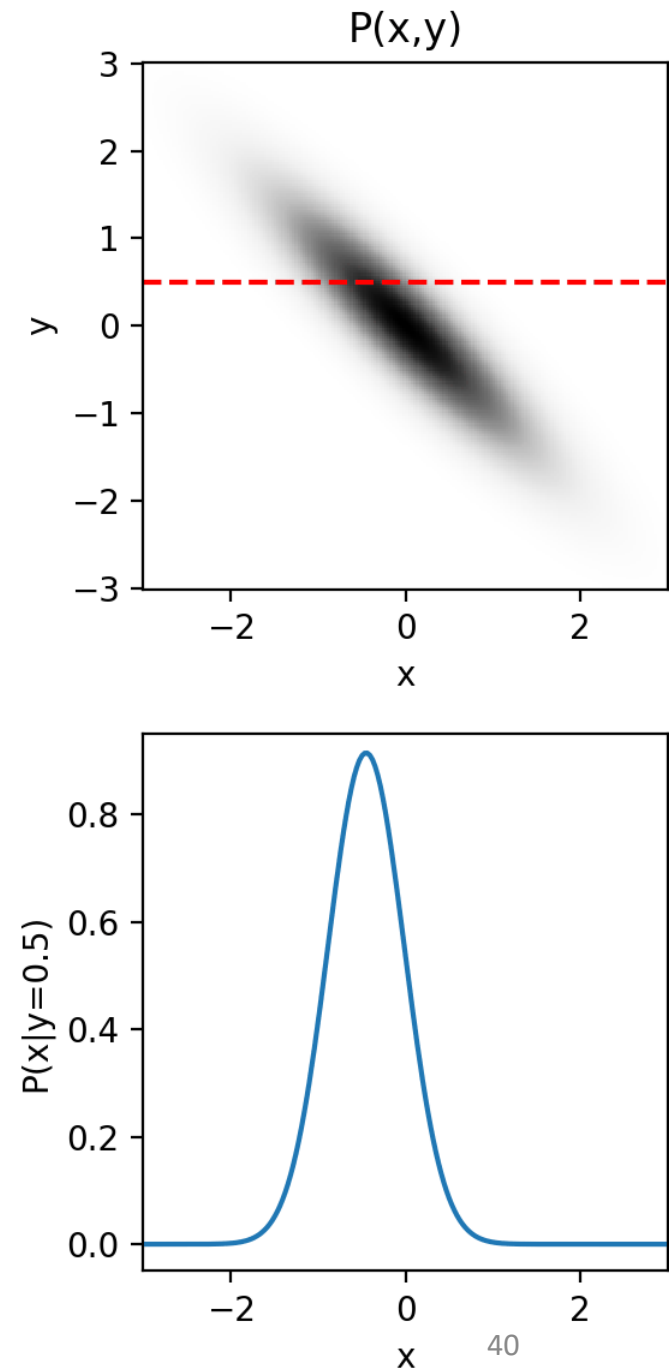
- We wish to predict the random variable Y_0 given data $\mathbf{Y}_n$, these will be correlated
- Assume joint distribution of prediction + data is multivariate normal:

$$\begin{pmatrix} Y_0 \\ \mathbf{Y}_n \end{pmatrix} \sim N_{1+n} \left[\begin{pmatrix} \mu_0 \\ \boldsymbol{\mu}_n \end{pmatrix}, \sigma^2 \begin{pmatrix} R_{11} & \mathbf{R}_{12} \\ \mathbf{R}_{21} & \mathbf{R}_{22} \end{pmatrix} \right]$$

- Conditional distribution of known data has mean and covariance matrix given by:

$$E(Y_0) = \mu_0 + \mathbf{R}_{12} \mathbf{R}_{22}^{-1} (\mathbf{Y}_n - \boldsymbol{\mu}_n)$$

$$\text{Var}(Y_0) = \sigma^2 (R_{11} - \mathbf{R}_{12} \mathbf{R}_{22}^{-1} \mathbf{R}_{21})$$



Conditioning multivariate normals

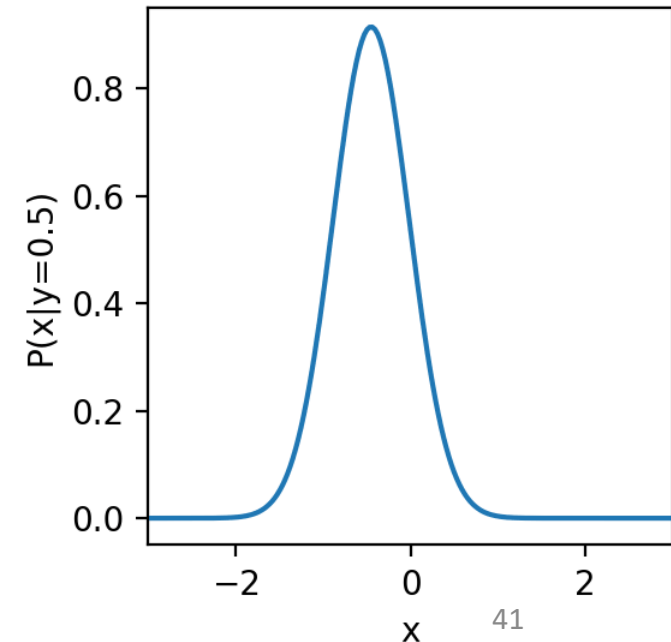
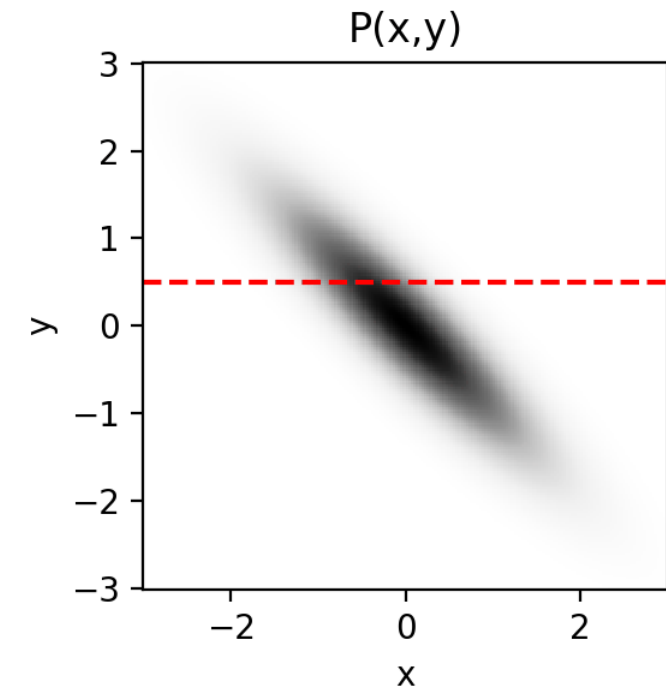
- Assume joint distribution of prediction + data is multivariate normal:

$$\begin{pmatrix} Y_0 \\ \mathbf{Y}_n \end{pmatrix} \sim N_{1+n} \left[\begin{pmatrix} \mu_0 \\ \boldsymbol{\mu}_n \end{pmatrix}, \sigma^2 \begin{pmatrix} R_{11} & \mathbf{R}_{12} \\ \mathbf{R}_{21} & \mathbf{R}_{22} \end{pmatrix} \right]$$

- For bivariate example, (x,y) given y

$$\begin{aligned} E(x|y) &= \mu_x + \rho y \\ \text{Var}(x|y) &= (1 - \rho^2)\sigma^2 \end{aligned}$$

- Where ρ is the correlation ($-1 < \rho < 1$)
- How do we decide on a correlation value?



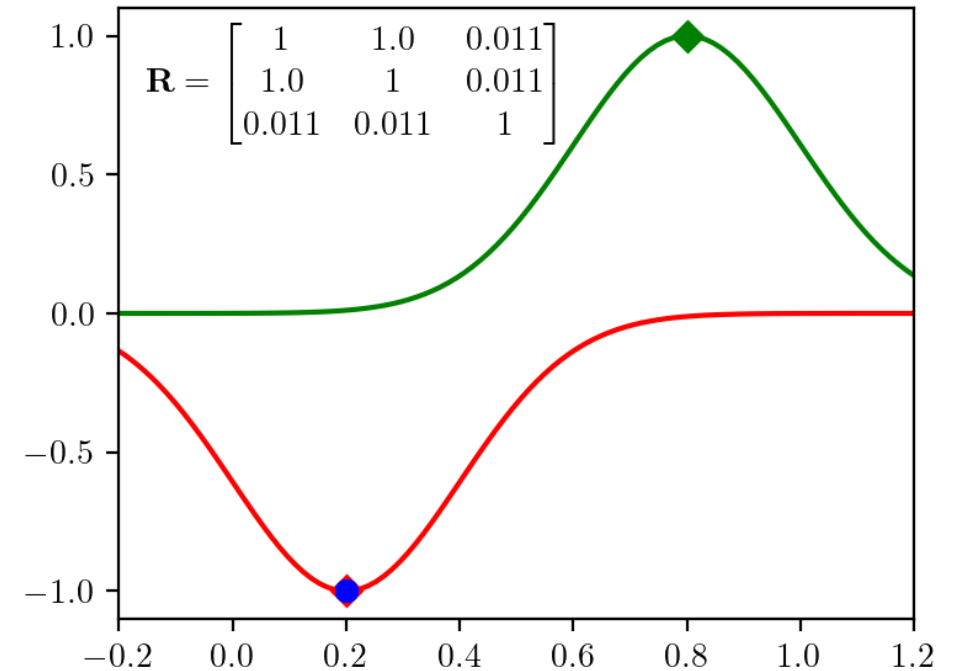
Gaussian Processes

$$\underline{\Sigma} = \sigma^2 \begin{bmatrix} R(x_0 - x_0) & R(x_1 - x_0) & \cdots & R(x_N - x_0) \\ R(x_0 - x_1) & R(x_1 - x_1) & \cdots & R(x_N - x_1) \\ \vdots & \vdots & \ddots & \vdots \\ R(x_0 - x_N) & R(x_1 - x_N) & \cdots & R(x_N - x_N) \end{bmatrix}$$

- Gaussian processes use the conditional distribution to predict unobserved points
- Mean function (usually 0) is linear fitting function, $\mu(x) = \sum_i \theta_i f_i(x)$
- Correlation matrix defined by 'kernel' function $R(d)$

$$Y(x) = \mu(x) + Z(\mu = 0, R(d))$$

- Correlation function quantifies how correlated points in space are
- How do you pick kernel free parameters...



$$E(Y_0) = R_{12} R_{22}^{-1} Y_n$$

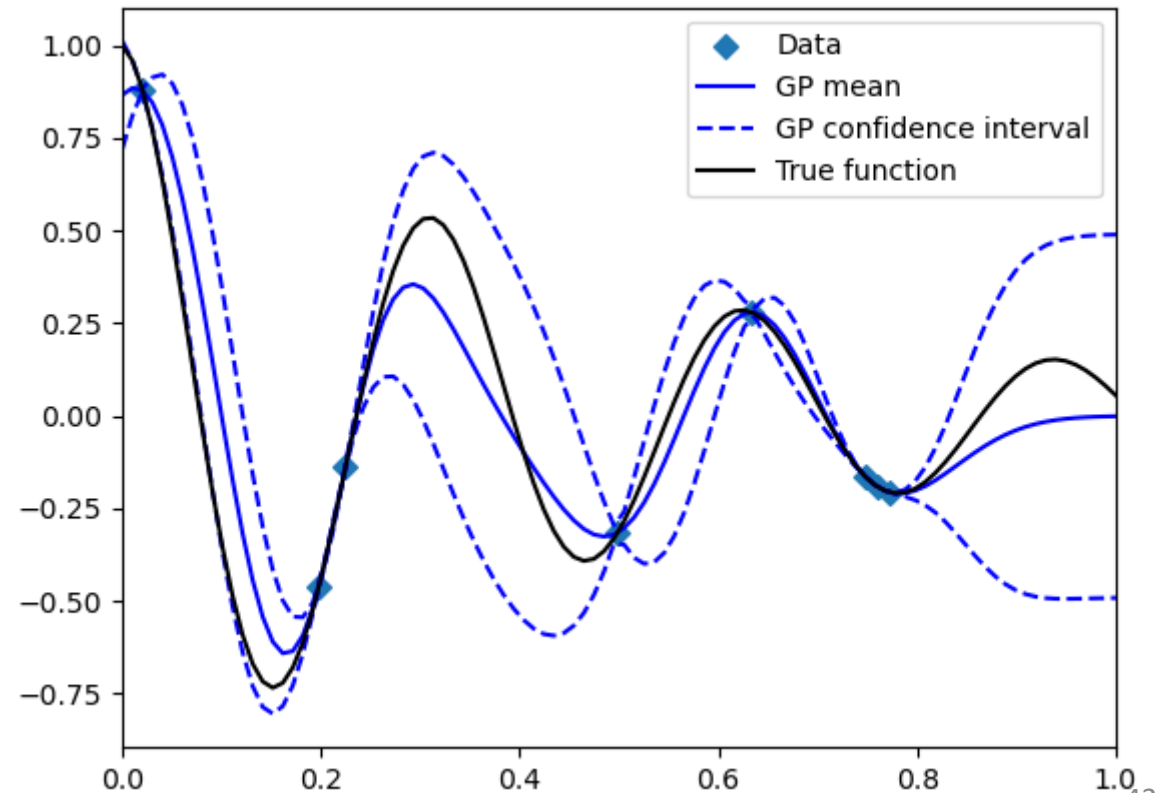
$$R(d, l) = \exp \left[-\frac{1}{2} \left(\frac{d}{l} \right)^2 \right]$$

Gaussian Processes: Example

```
16 def correlation_function(x1,x2,theta):
17     """ Correlation function or kernel of the GP, we will use a Gaussian with length scale theta """
18     return np.exp(-0.5*((x1-x2)/theta)**2)
19
20 def correlation_matrix(x,theta):
21     """ Construct correlation matrix using correlation function """
22     x1,x2 = np.meshgrid(x,x)
23     corr = correlation_function(x1,x2,theta)
24     return corr
25
26 def sigma2_MLE(corr,y):
27     """ Maximum Likelihood estimation of vertical length scale sigma
28     | see pg 65-66 of Santner, Williams and Notz """
29     n = y.shape[0]
30     return np.dot(y,np.matmul(np.linalg.inv(corr),y))/n
31
32 def neg_log_likelihood(theta,x,y):
33     """ Negative Log Likelihood of Gaussian process for a given choice of length scale theta
34     | see pg 65-66 of Santner, Williams and Notz """
35     n = x.shape[0]
36     corr = correlation_matrix(x,theta)
37     sigma2 = sigma2_MLE(corr,y)
38     log_l = n*np.log(sigma2)+np.log(np.linalg.det(corr))
39     return log_l
40
41 def conditional_distribution(x_trial,x_known,y_known,sigma,theta):
42     """ Calculate p(x_trial | x_known,y_known) for given values of sigma and theta """
43     x_total = np.concatenate((x_trial,x_known),axis=0)
44     N_trial = x_trial.shape[0]
45     corr = sigma**2*correlation_matrix(x_total,theta)
46     corr_11 = corr[:N_trial,:N_trial]
47     corr_12 = corr[:N_trial,N_trial:]
48     corr_21 = corr_12.T
49     corr_22 = corr[N_trial:,N_trial:]
50     # Note unconditional mu = 0
51     corr_22_inv = np.linalg.inv(corr_22)
52     mu_cond = np.dot(np.matmul(corr_12,corr_22_inv),y_known)
53     sig_cond = corr_11 - np.matmul(corr_12,np.matmul(corr_22_inv,corr_21))
54     return mu_cond,sig_cond
55
56 res = minimize(neg_log_likelihood,0.1,args=(X_known,Y_known))
57
58 theta_opt = res.x[0]
59 corr_opt = correlation_matrix(X_known,theta_opt)
60 sigma_opt = np.sqrt(sigma2_MLE(corr_opt,Y_known))
61
62 X_trial = np.linspace(0.0,1.0,100)
63
64 mu_cond,sig_cond = conditional_distribution(X_trial,X_known,Y_known,sigma_opt,theta_opt)
```

Kernel parameters found by maximal likelihood estimation given known data

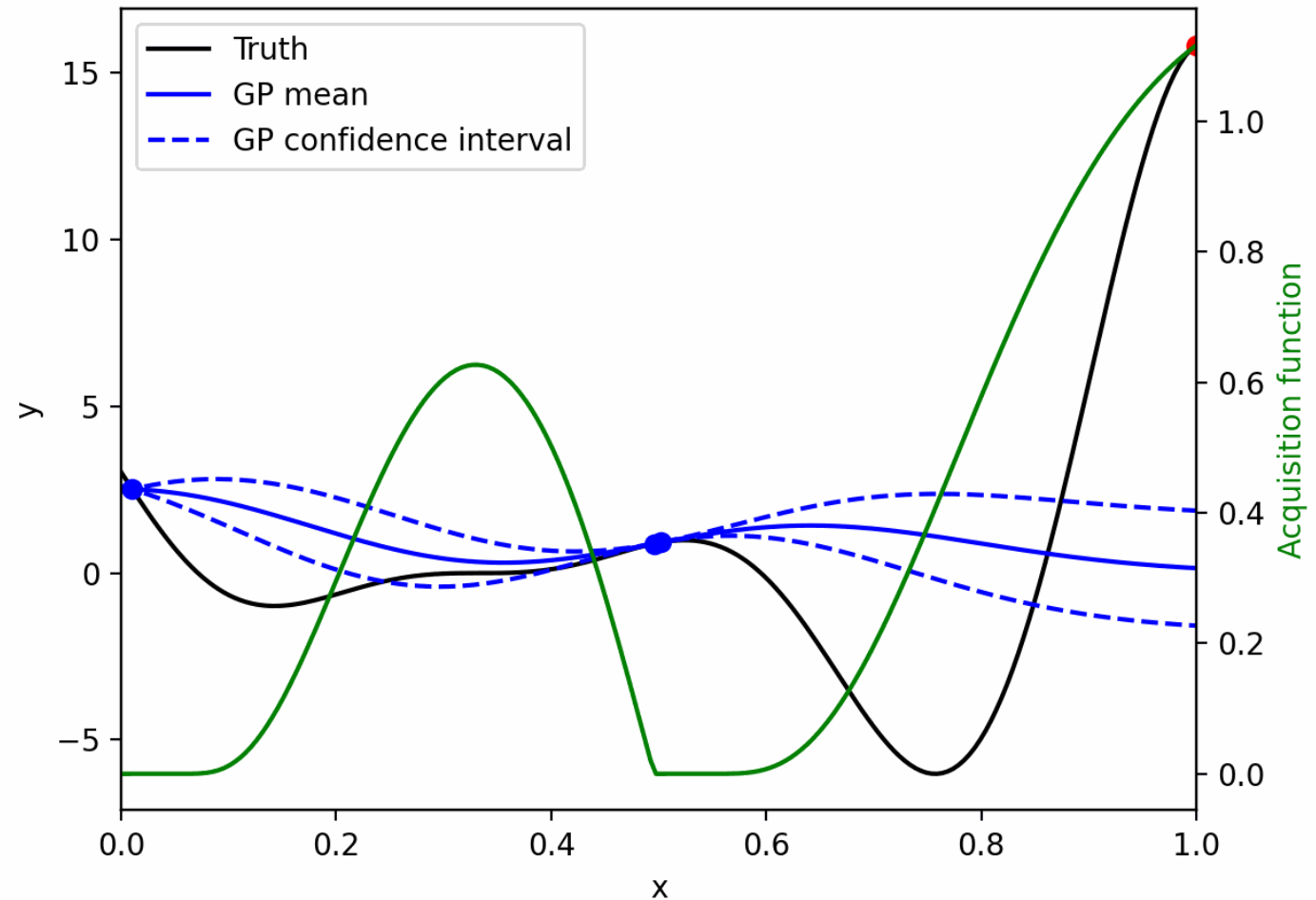
Python libraries: *Gpy*, *sklearn*, *GPyTorch*



Using Gaussian Process: Bayesian Optimisation

One canonical use of Gaussian Processes (GP) is in Bayesian optimisation:

1. From sample points train GP
2. Optimise an 'acquisition function' to find next sample point
3. Return to step 1 until global optimum found



Using Gaussian Process: Bayesian Optimisation

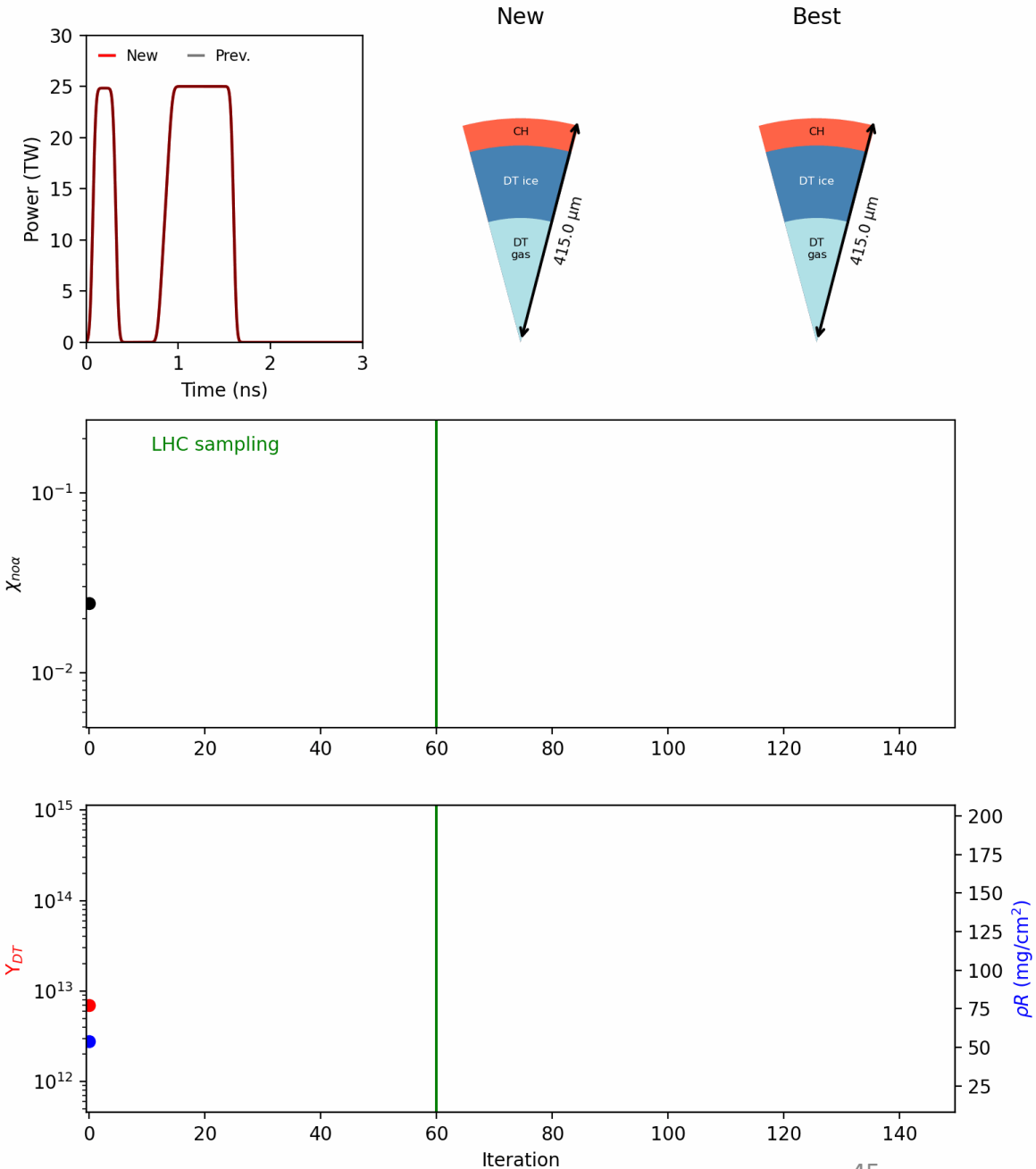
In practice:



Can be used to optimise
expensive simulations

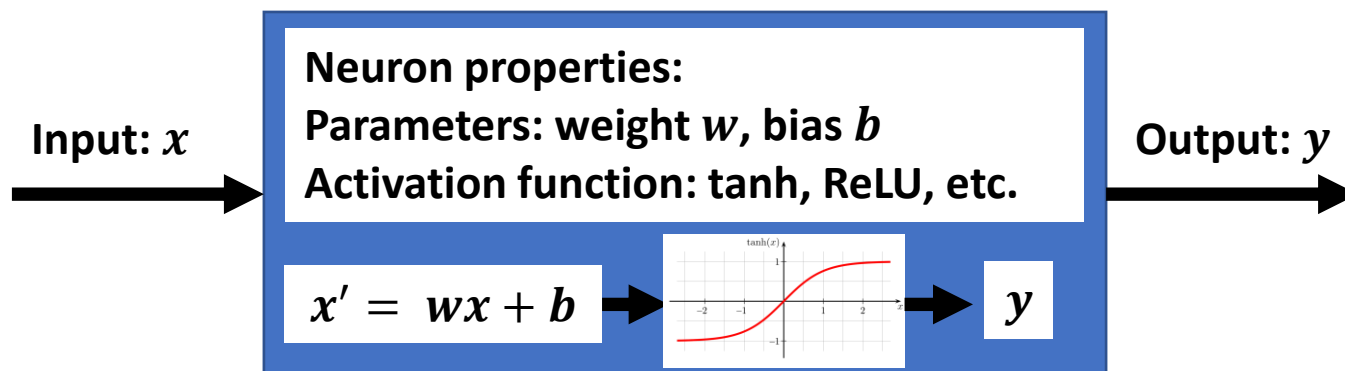
Usually start with a number of
pseudo-random samples before
training GP and starting BO

Useful packages: BOTorch



Neural Networks

- Neural networks are the canonical AI model
- They are black-box functions with many, many learnable parameters, *ChatGPT 175 billion*
- Many flavours, we will discuss the multi-layer perceptron (MLP)
- Basic unit is the neuron:

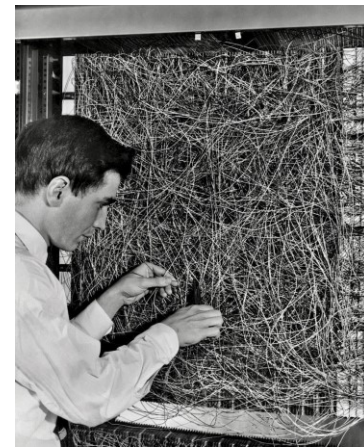
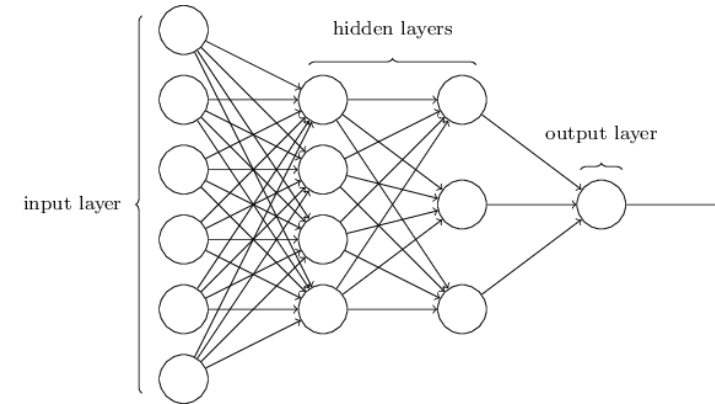


Multi-Layer Perceptron

- Stack neurons into layers and *fully-connect* layers
- Define loss function between predicted and real outputs e.g. mean-squared-error (MSE)
- Update neuron parameters by gradient descent (or variations of it)
- It is just regularised non-linear least squares again!

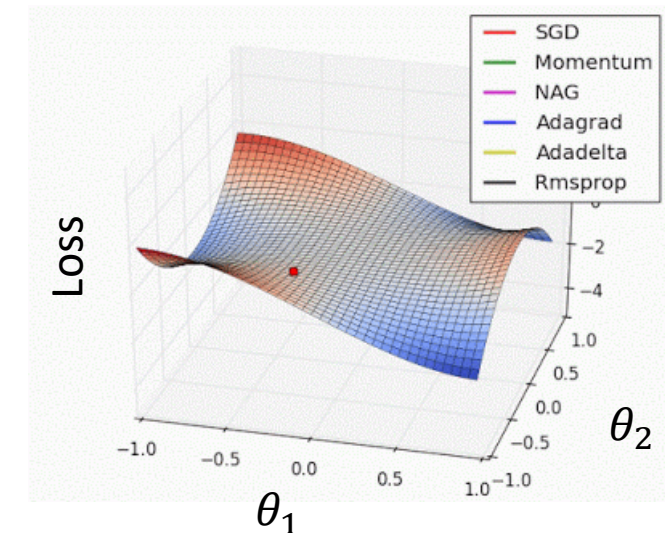
$$d\theta = \alpha \cdot J^T(y - f(x, \theta))$$

Learning rate Jacobian found by AD Neural network



*Rosenblatt with
the Mark 1
Perceptron*

Various optimisers

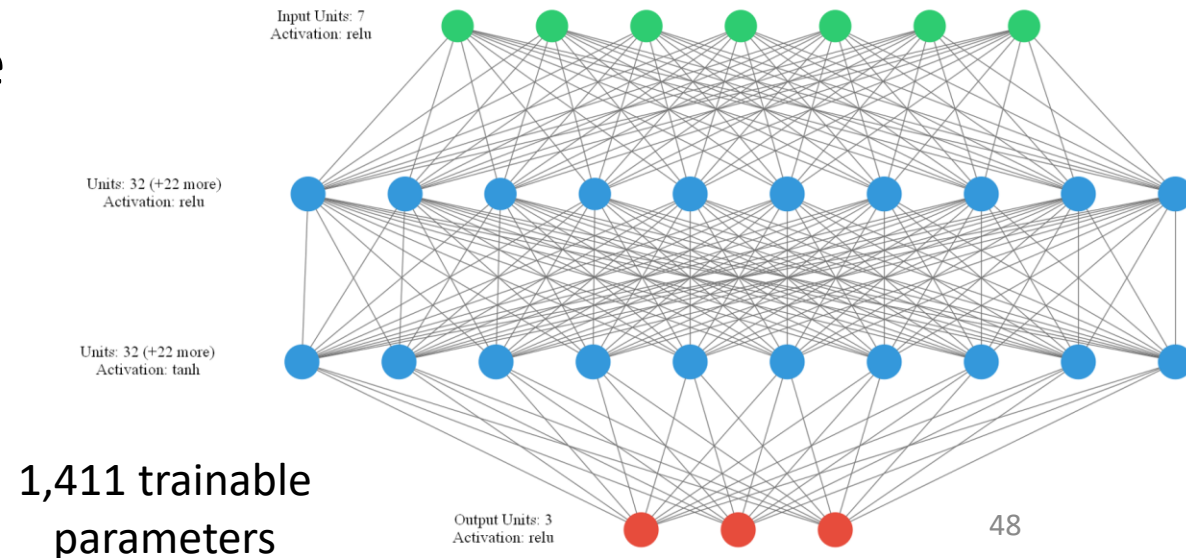
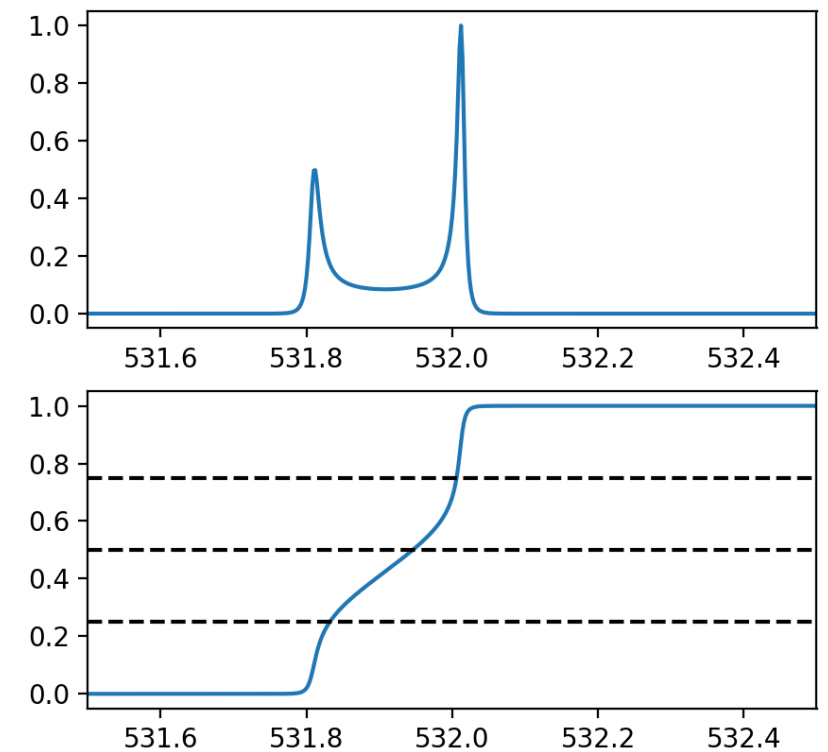


Neural Networks: Example

- Learn relationship between Si IAW Thomson scattering features and physical parameters

$$\theta = (n_e, \mu, T_e, T_i, Z, v_{fi}, v_{fe})$$
$$\mathbf{y} = (\Delta\lambda_m, \Delta\lambda_h - \Delta\lambda_l, a_\lambda)$$

- Selection of the NN architecture can be human judgement or algorithmic
 - Tweaking of model parameters becomes tweaking of hyper-parameters
- Data 'featurisation' is often key in producing a good model



Neural Networks: Example

- Learn relationship between Si IAW Thomson scattering features and physical parameters

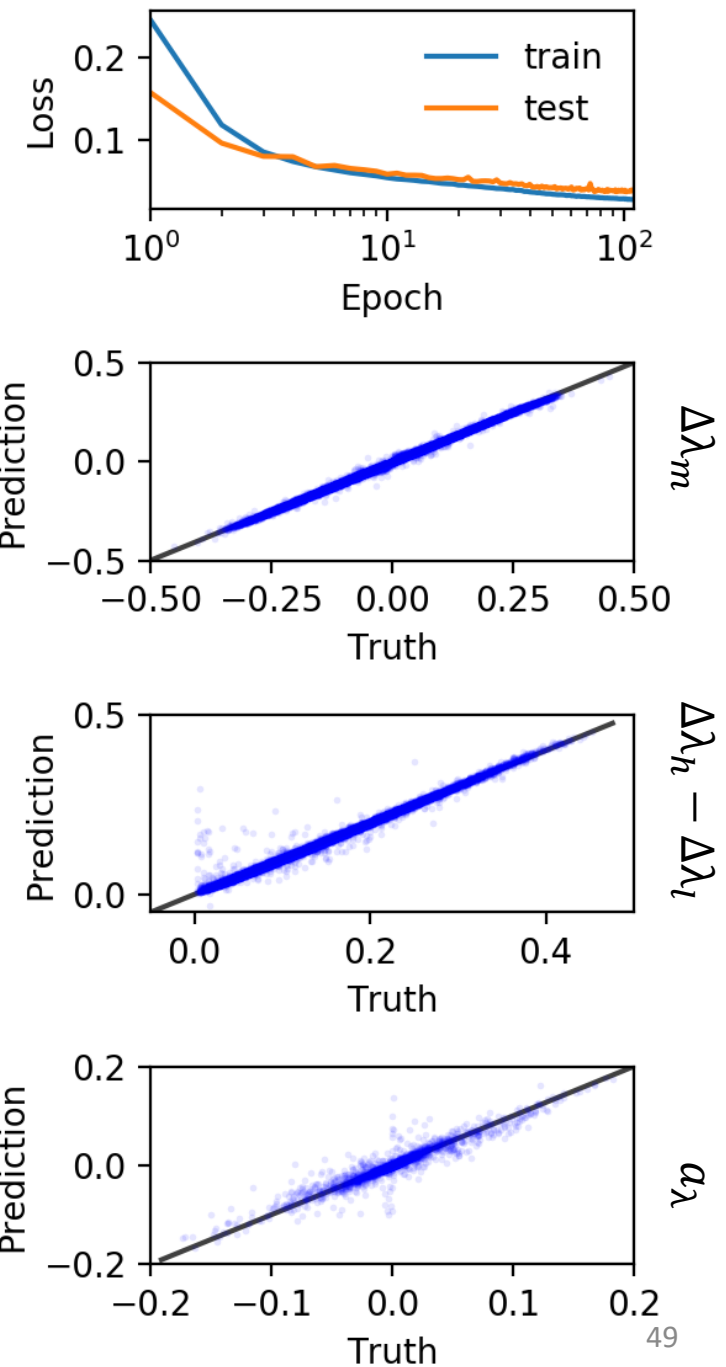
$$\boldsymbol{\theta} = (n_e, \mu, T_e, T_i, Z, v_{fi}, v_{fe})$$
$$\boldsymbol{y} = (\Delta\lambda_m, \Delta\lambda_h - \Delta\lambda_l, a_\lambda)$$

- Deep learning libraries:
keras/tensorflow, sklearn, pytorch

```
# define the keras model
model = Sequential()
model.add(InputLayer(input_shape=(input_size,)))
model.add(Dense(32, input_shape=(input_size,), activation='relu'))
model.add(Dense(32, activation='tanh'))
model.add(Dense(output_size))

model.summary()

model.compile(loss='mean_squared_error', optimizer='adam', metrics=['accuracy'])
history = f1
```



Neural Networks: Example

- Learn relationship between Si IAW Thomson scattering features and physical parameters

$$\boldsymbol{\theta} = (n_e, \mu, T_e, T_i, Z, v_{fi}, v_{fe})$$
$$\boldsymbol{y} = (\Delta\lambda_m, \Delta\lambda_h - \Delta\lambda_l, a_\lambda)$$

- Deep learning libraries:
*keras/tensorflow, sklearn, **pytorch***

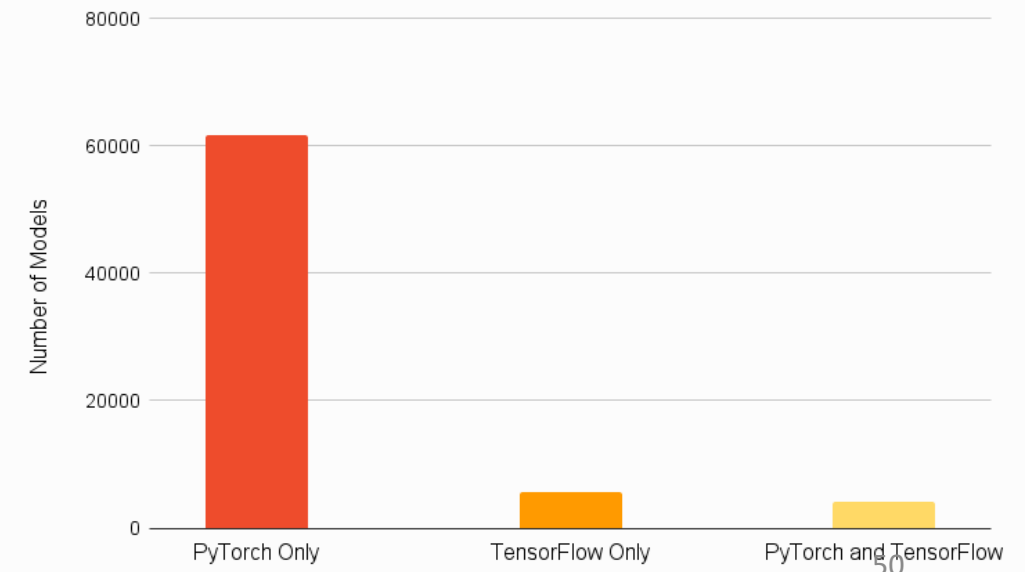
```
106
107 class MLP(nn.Module):
108     def __init__(self, input_size, output_size):
109         super(MLP, self).__init__()
110         self.layers = nn.Sequential(
111             nn.Linear(input_size, 32),
112             nn.ReLU(),
113             nn.Linear(32, 32),
114             nn.Tanh(),
115             nn.Linear(32, output_size)
116         )
117
118     def forward(self, x):
119         return self.layers(x)
120
```

- Learn relationship between Si IAW Thomson scattering features and physical parameters

$$\boldsymbol{\theta} = (n_e, \mu, T_e, T_i, Z, v_{fi}, v_{fe})$$
$$\boldsymbol{v} = (\Delta\lambda_m, \Delta\lambda_h - \Delta\lambda_l, a_\lambda)$$

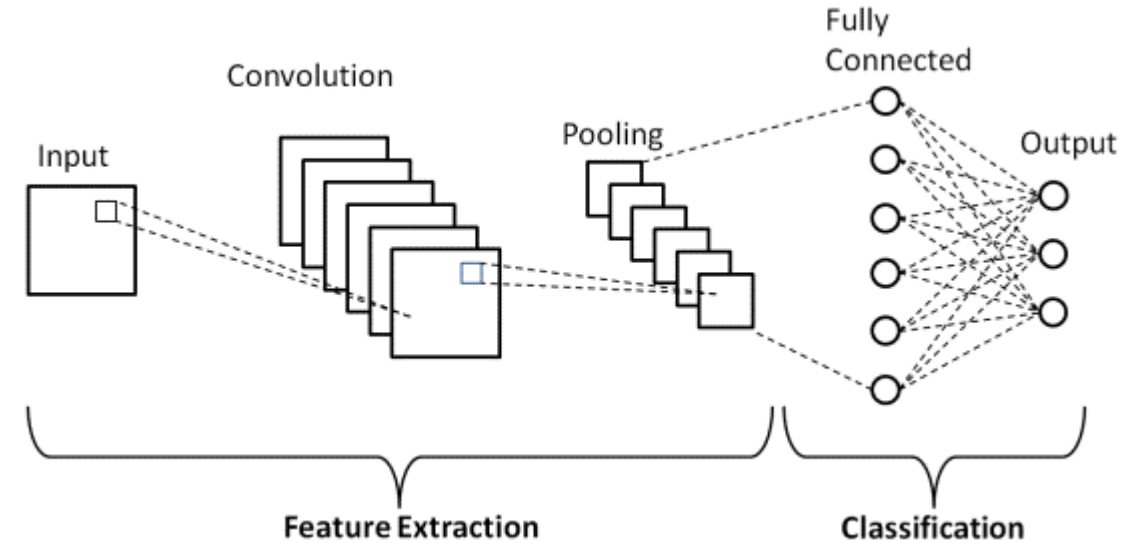
$\Delta\lambda_m$

Number of Models on HuggingFace



Other architectures – Convolutional Neural Networks

- CNNs use trainable convolutional kernels rather than taking in the whole input
 - Reduces the number of parameters massively



Example:

- Information is locally stored in an image
- Canonical convolutional masks can be used to detect edges

The example shows a 6x6 input grid with a 3x3 kernel (mask) applied to it. The kernel values are 1, 0, -1 in each row. The result is a 4x4 output grid where the top-left cell contains the value -5.

3 ₁	0 ₀	1 ₁	2	7	4
1 ₁	5 ₀	8 ₁	9	3	1
2 ₁	7 ₀	2 ₁	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

6 × 6

1	0	-1
1	0	-1
1	0	-1

3 × 3

* =

-5			

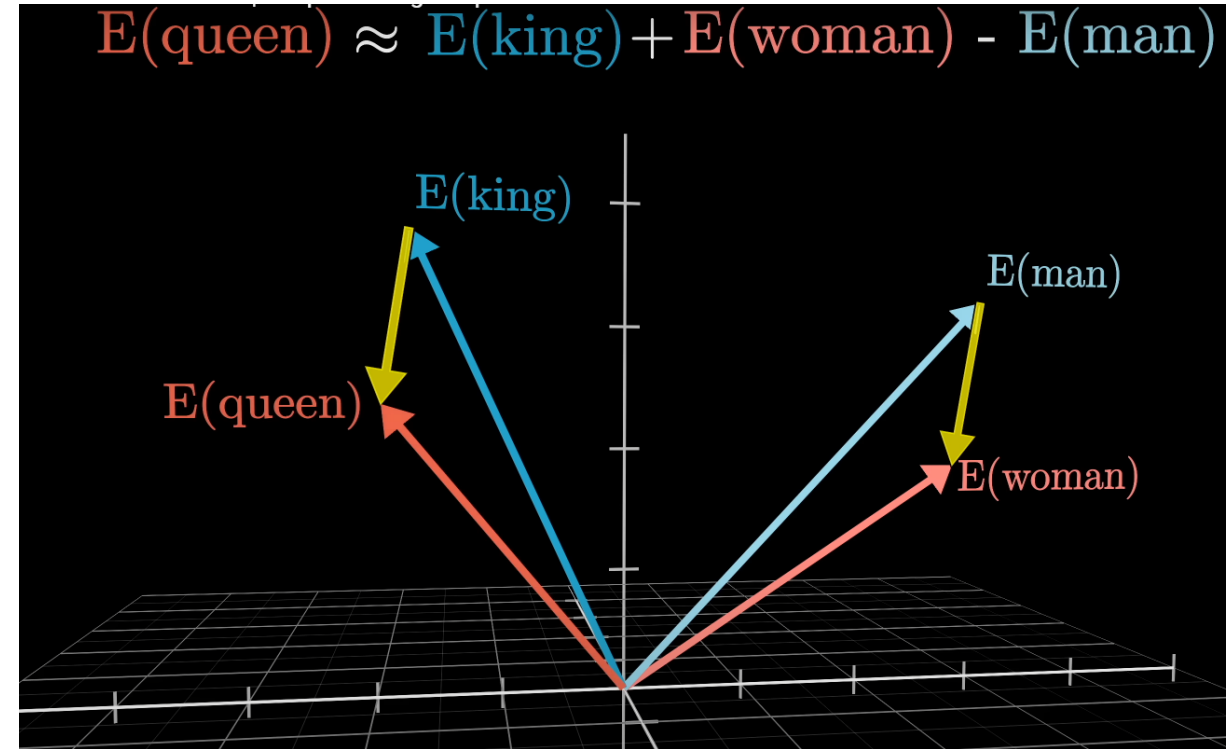
4 × 4

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

Other architectures – Transformers & Attention

- Transformers are basis of modern large language models
- Sequences are split into tokens which are represented by unique vectors in high dimensional “embedding” space

Embedding vectors encode semantic meaning:



GPT 3 embedding space is 12288-dimensional

Those interested: 3 Brown 1 Blue video is excellent

Other architectures – Transformers & Attention

- Transformers are basis of modern large language models
- Sequences are split into tokens which are represented by unique vectors in high dimensional “embedding” space
- “Attention” mechanism is used to find updates to these embeddings that reflect the context

Those interested: 3 Brown 1 Blue video is excellent

Attention is all you need

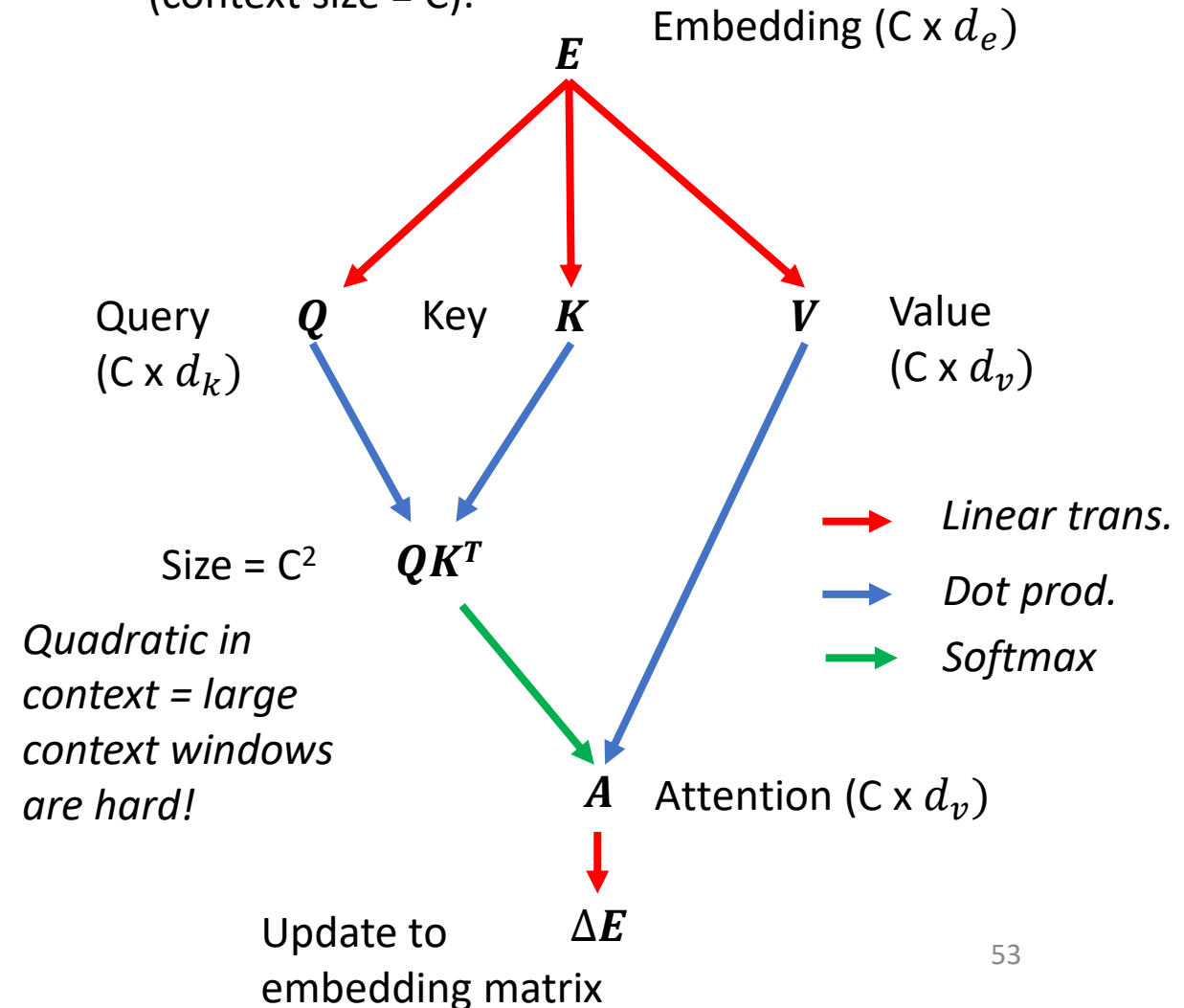
[A Vaswani, N Shazeer, N Parmar...](#) - Advances in neural ..., 2017 - proceedings.neurips.cc

... to attend to **all** positions in the decoder up to and including that position. **We need** to prevent

... **We** implement this inside of scaled dot-product **attention** by masking out (setting to $-\infty$) ...

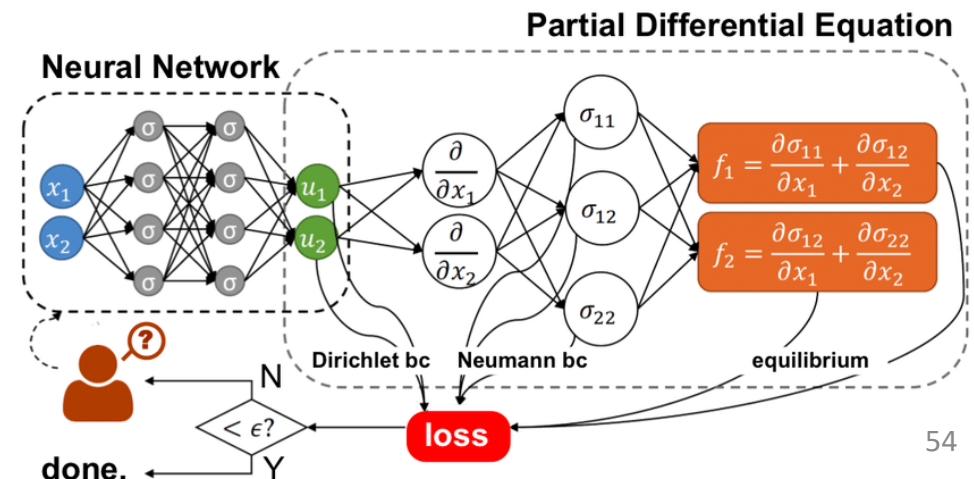
☆ Save 📄 Cite Cited by 202050 Related articles All 70 versions 🔗

Simplified view of single head of attention
(context size = C):



Loss terms for neural networks

- The most common are ‘prediction metrics’ such as mean squared error
- Regularisers on weights and biases, common example is ‘L2-norm’ which is the same as Tikhonov, $\lambda|\boldsymbol{\theta}|^2$
 - λ is now hyper-parameter like learning rate, α
- More exotic terms are being introduced, like physics informed neural networks
 - AD allows derivative terms to be added to the loss



Physics Informed Neural Networks (PINNs)

- What if we know some physical law we want our NN to respect?
- PINN = NN learns (approximate) solution to PDE/ODE:

Want to train NN $y_{NN}(x, t)$ to match some data while respecting physical law g_{PDE} (or boundary condition)

$$y_{NN}(x, t) \rightarrow y(x, t)$$

$$g_{PDE} := \frac{\partial y}{\partial t} + f(y; x, t) = 0$$

- Include a loss term which ensure g_{PDE} is (approximately) respected:

$$L = L_y + L_g$$

Physics Informed Neural Networks

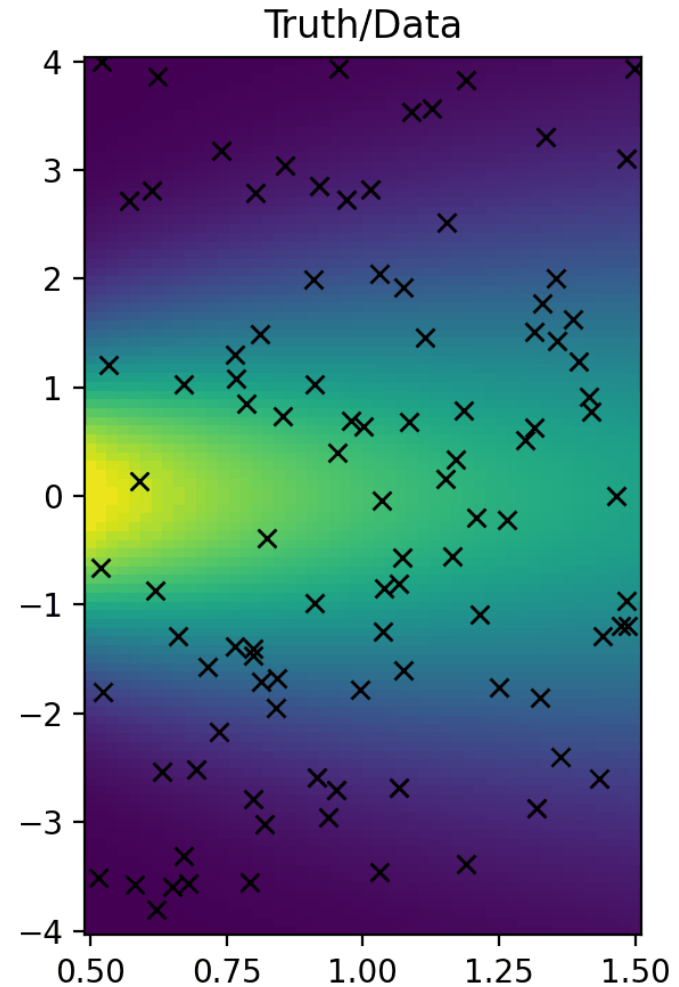
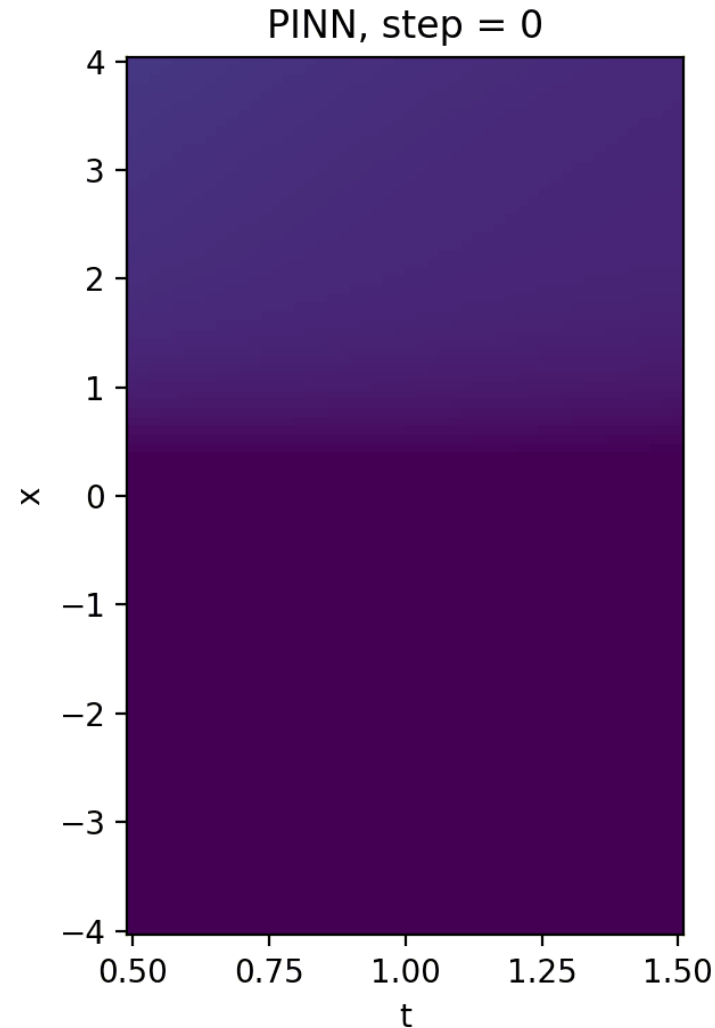
- For example, diffusion:

$$y_{NN}(x, t) \rightarrow y(x, t)$$

$$g_{PDE} := \frac{\partial y}{\partial t} - D \frac{\partial^2 y}{\partial x^2} = 0$$

$$\begin{aligned} L &= \frac{1}{N} \sum (y_{NN} - y)^2 \\ &+ \frac{1}{N} \sum \left(\frac{\partial y_{NN}}{\partial t} - D \frac{\partial^2 y_{NN}}{\partial x^2} \right)^2 \end{aligned}$$

Computed using automatic differentiation of NN



Use of AI in research

- Large Language Models (LLMs) are effective research tools
 - If you apply a bit of caution!
- Effective use cases:
 - Code generation
 - Code translation (fortran to Python, etc.)
 - Agent based workflows
 - Drafting technical documents
 - Research questions
- Only truly effective if you can assess the output
 - Don't use it to turn off your brain, try the opposite!



Empowering the next generation of developers.
Eligible university students get
one free year of Cursor Pro.

Apply →

Use of AI in research

- For research (*or maybe all!*) questions be sure to use reasoning/thinking models

ChatGPT Instant

I am interesting in solving Poissons equation in spherical coordinates for the potential causes by a central ion (charge Ze) surrounded by uniform density electrons which neutralise this charge within a radius R .

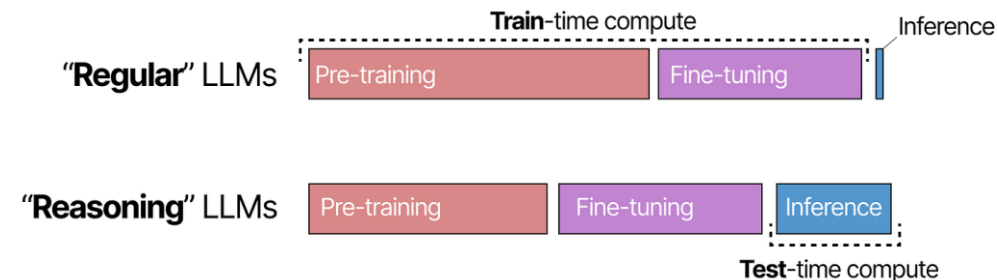
What is the solution for the potential as a function of radius? Please just provide this solution, do not give derivation.

For a spherically symmetric system with a central ion of charge $+Ze$ surrounded by a uniform electron charge density that exactly neutralizes the ion within radius R , the electrostatic potential $\phi(r)$ is:

$$\phi(r) = \begin{cases} \frac{Ze}{8\pi\epsilon_0 R} \left(3 - \frac{r^2}{R^2} \right), & r \leq R, \\ \frac{Ze}{4\pi\epsilon_0 r}, & r \geq R. \end{cases}$$

This potential is continuous and differentiable at $r = R$, and goes to zero as $r \rightarrow \infty$.

Wrong ❌



ChatGPT Thinking

I am interesting in solving Poissons equation in spherical coordinates for the potential causes by a central ion (charge Ze) surrounded by uniform density electrons which neutralise this charge within a radius R .

What is the solution for the potential as a function of radius? Please just provide this solution, do not give derivation.

Thought for 58s >

For total neutrality inside radius R (uniform electron charge density $\rho_e = -\frac{3Ze}{4\pi R^3}$) and $\phi(\infty) = 0$, the spherically symmetric solution is

$$\phi(r) = \begin{cases} \frac{Ze}{4\pi\epsilon_0} \left(\frac{1}{r} - \frac{3}{2R} + \frac{r^2}{2R^3} \right), & 0 < r \leq R, \\ 0, & r \geq R. \end{cases}$$

Right ✅

Use of AI in research

- Power of Command Line Interface (CLI) to remote LLMs
 - Gemini CLI (free), **Codex**, Claude, etc.
- Example: *Prompt: This repository contains code that implemented adaptative quadrature in JAX. It is written such that the quadrature function can be automatically differentiated by the application of Leibniz rule. In tests there is pytest code that confirms this behaviour by comparison of the numerical integrals and derivatives to exact results. **Can you write and implement a plan to write a new implementation that uses PyTorch instead of JAX and passes its own version of the same tests.***

```
• Updated Plan
  ↳ Added a PyTorch backend with custom autograd matching Leibniz rule, mirrored tests, and updated requirements. Now doing a quick sanity review and proposing next actions.
    ✓ Explore repo and tests
    ✓ Design Torch API + autodiff
    ✓ Implement Torch adaptive GTQ
    ✓ Add quadGtTorch package files
    ✓ Write Torch mirror tests
    ✓ Update requirements for torch
    □ Sanity review and next steps

- Worked for 3m 04s -

• All set. I added a PyTorch backend with custom autograd and tests that mirror the JAX ones.
```

Token usage: total=27,416 input=16,541 (+ 192,128 cached) output=10,875 (reasoning 6,592)

Machine Learning in SPC Research

- Examples of SPC community using ML:
 - “Using Differentiable Programming to Learn Closure Relations: An Example in Radiation Transport”, Plasma, Aidan Crilly
 - “Clustering Applied to PIC Simulation and Spacecraft Data”, Space, Cara Waters
 - “Dynamic Mode Decomposition of MHD simulations”, Space, Martin Archer
 - “Learning parameterisations and feature extraction in images”, Climate, Ed Gryspeerdt

Learning Closure Relations using Differentiable Programming: An Example in Radiation Transport

(Crilly et al Published @ NeurIPS ML 4 Physical Sciences Workshop 2023)

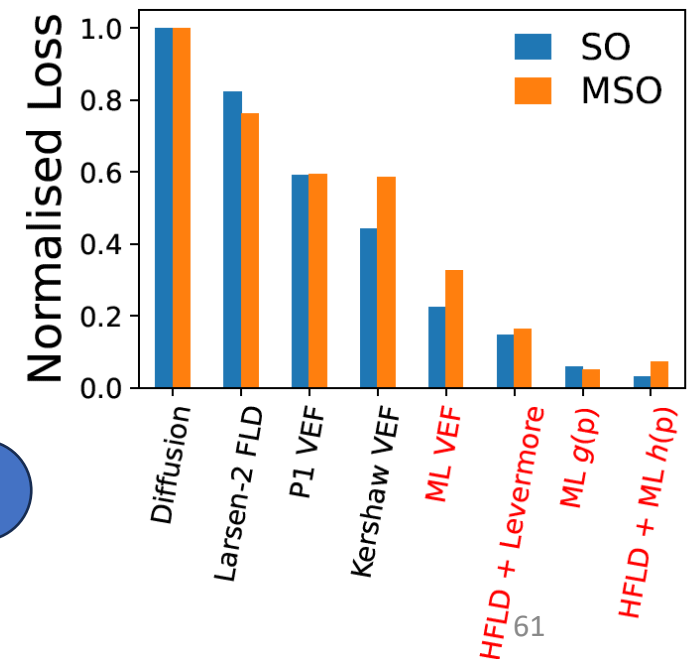
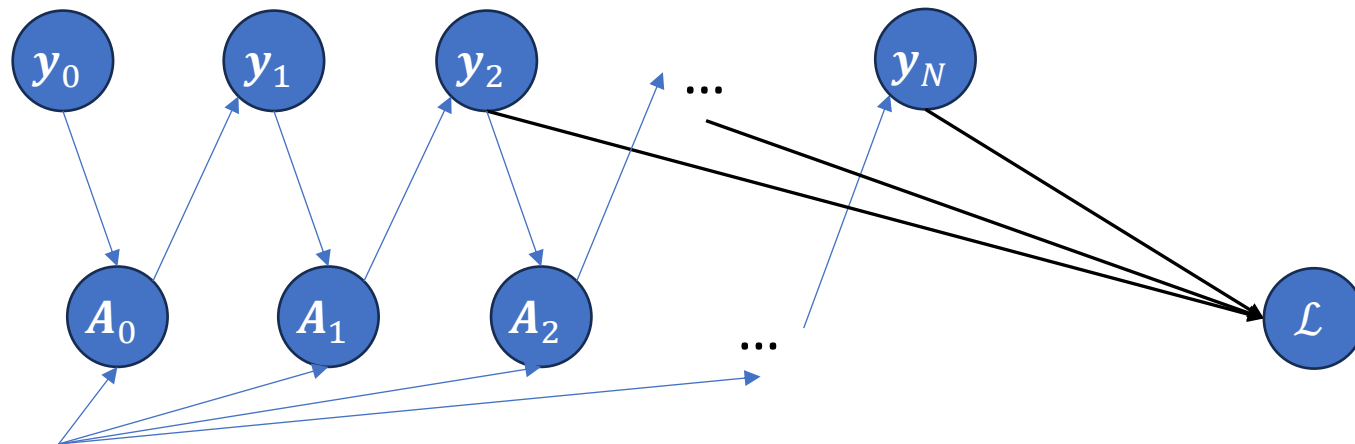
- Can back-propagate gradients through solutions to PDEs
- Allows closures to be learned from analytic test problems

Open-source code



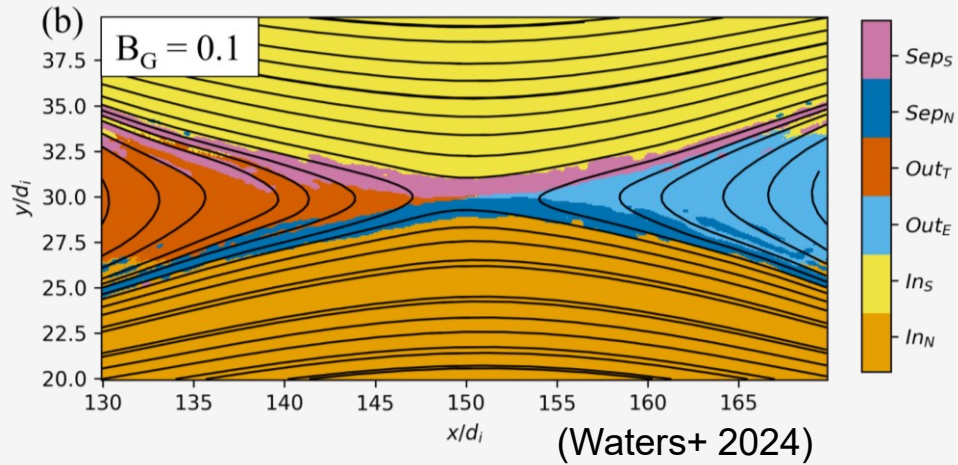
Initial
conditions

Model
parameters



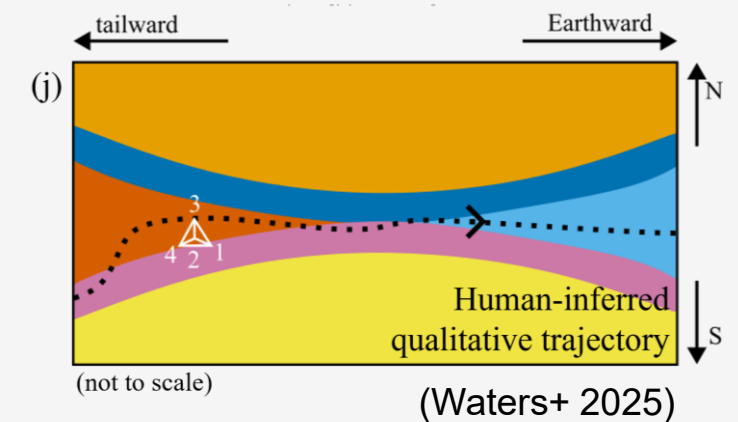
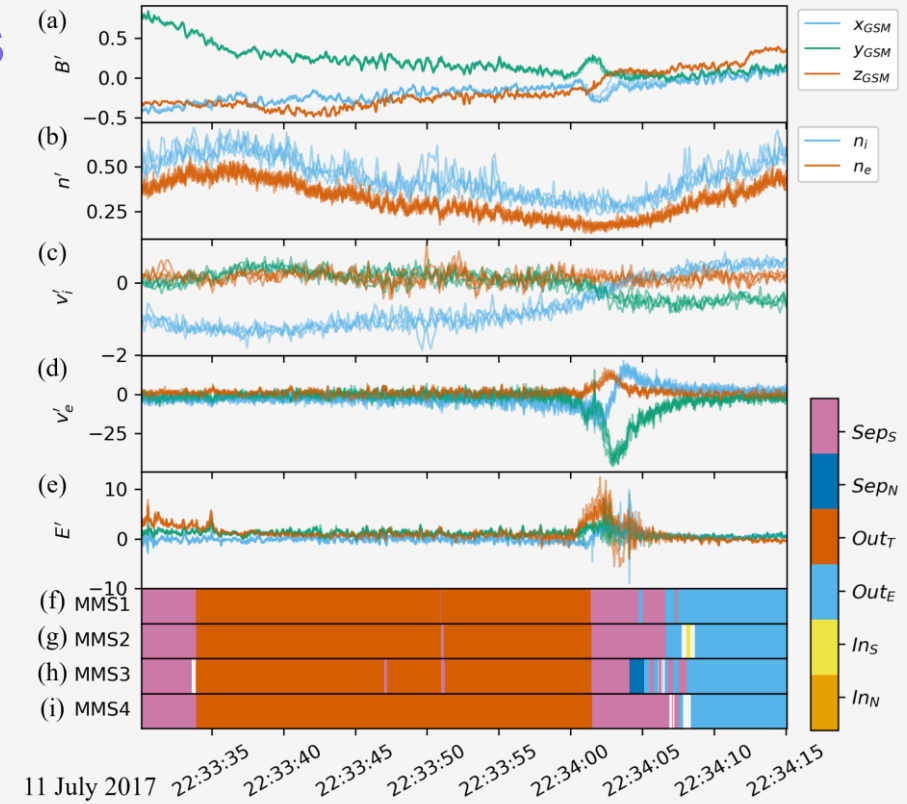
Clustering Applied to PIC Simulation and Spacecraft Data

ML for reproducible results and quantifiable biases



- K-means clustering applied (unsupervised) to simulation data
- Only plasma and field parameters provided, not position
- 14 parameters provided
- Identifies six regions as best quality of clustering
- These regions have physical meaning, without providing the model with any physical constraints

- Application to spacecraft time series allows for these regions to be directly related
- Quantitative comparison between spacecraft and simulation data
- Can infer a trajectory taken by a spacecraft from the classifications
- Benefit of discrete regions – can carry out statistical work



Dynamic Mode Decomposition (DMD)

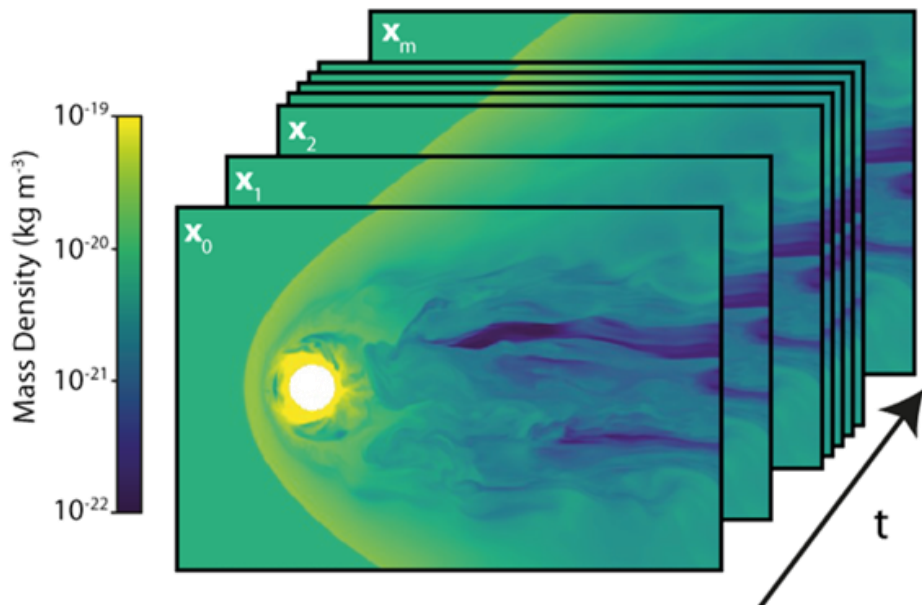
Spatiotemporal Dimensionality Reduction Algorithm

Space and **time** behave differently, but methods like PCA treat them the same

DMD decomposes data into **quasi-stationary dynamic modes** with **fixed frequency & growth rate**

a) Simulation data

State vector of physical quantities for each snapshot



Arrange state vectors into tensors

$$\mathbf{X}_1 = \begin{pmatrix} | & | & | & | \\ \mathbf{x}_0 & \mathbf{x}_1 & \cdots & \mathbf{x}_{m-1} \\ | & | & | & | \end{pmatrix}, \quad \mathbf{X}_2 = \begin{pmatrix} | & | & | & | \\ \mathbf{x}_1 & \mathbf{x}_2 & \cdots & \mathbf{x}_m \\ | & | & | & | \end{pmatrix}$$

b) DMD algorithm

Koopman operator
(data-driven approximation)

Eigendecomposition

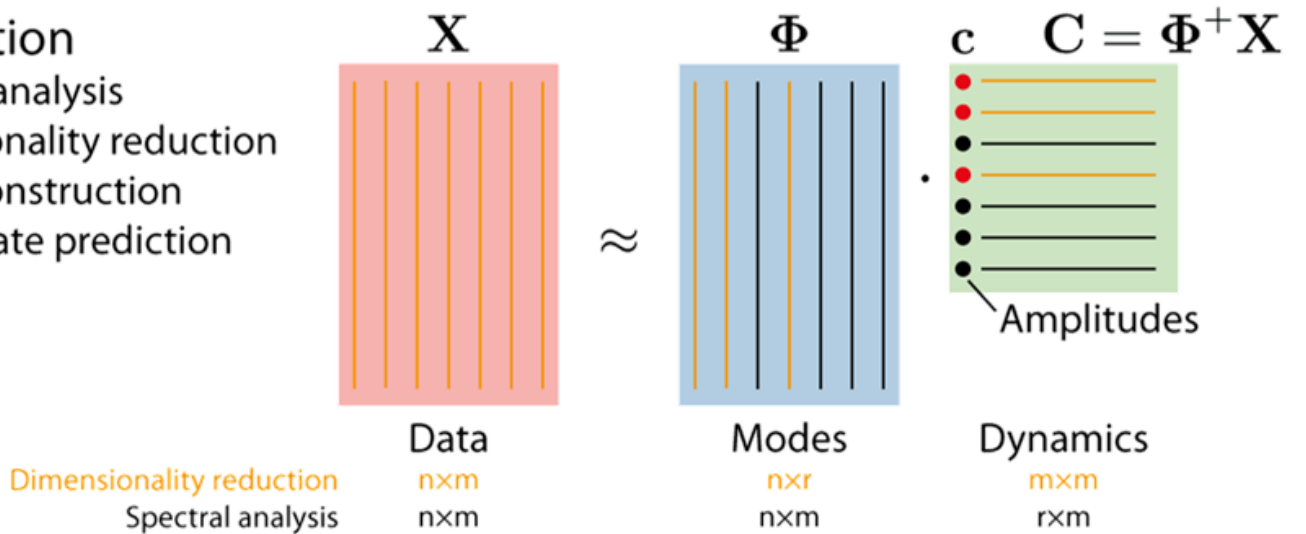
- Factorization
- Spectral analysis
 - Dimensionality reduction
 - Data reconstruction
 - Future state prediction

Model $\mathbf{x}_{k+1} = \mathbf{F}(\mathbf{x}_k) \approx \mathbf{A}\mathbf{x}_k$

$\mathbf{A} = \mathbf{X}_2 \mathbf{X}_1^+$ — Pseudoinverse (least-squares regression) — Koopman operator

$\mathbf{A}\Phi = \Lambda\Phi$

Eigenvectors (spatial) — Eigenvalues (temporal)

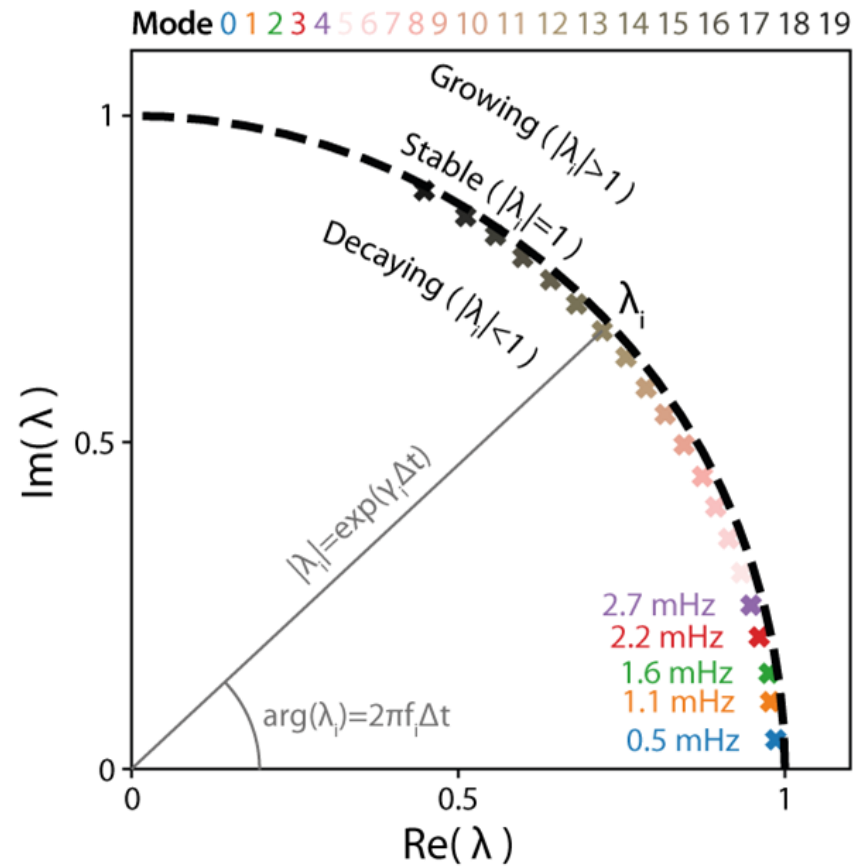


Dynamic Mode Decomposition (DMD)

Gorgon simulation of non-linear magnetopause Kelvin-Helmholtz waves

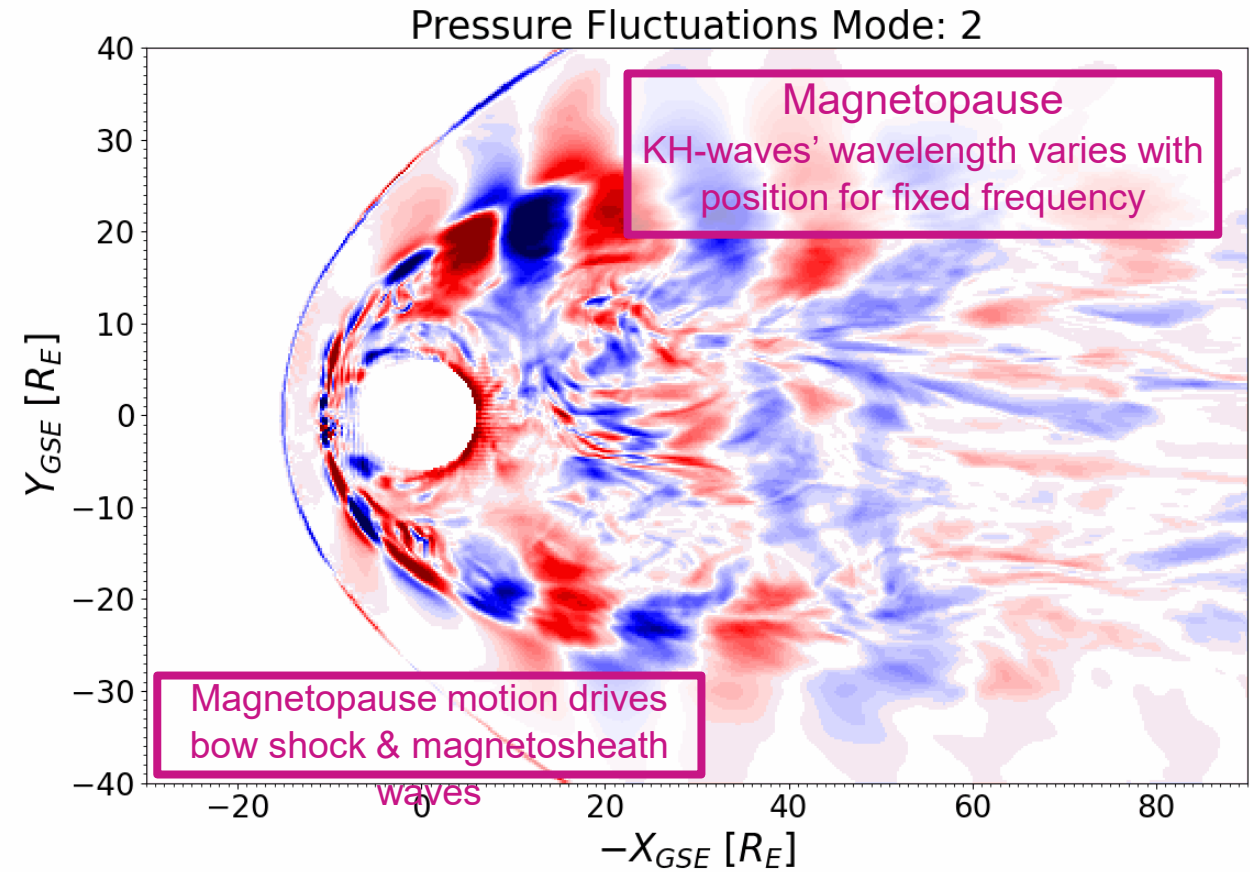
c) Eigenvalues

Mode quasi-stationary temporal evolution



d) Eigenvectors

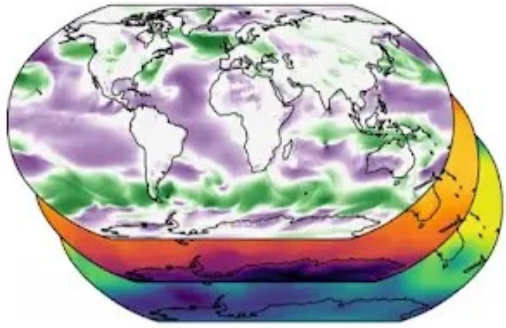
Mode spatial distribution



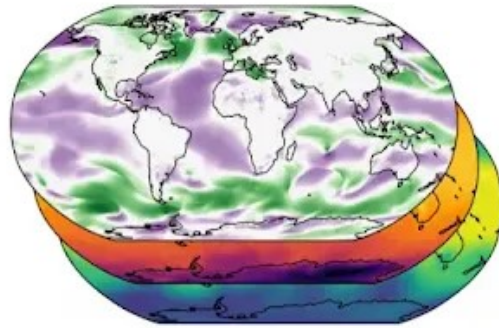
A superposition of KH-waves which vary in amplitude and wavelength spatially along boundary

Machine learning the weather

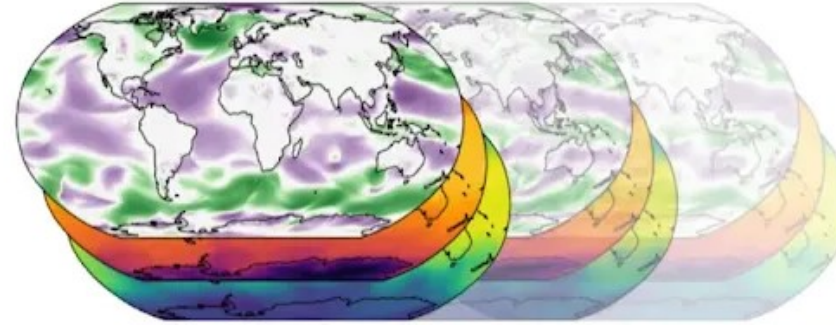
a) Input weather state



b) Predict the next state

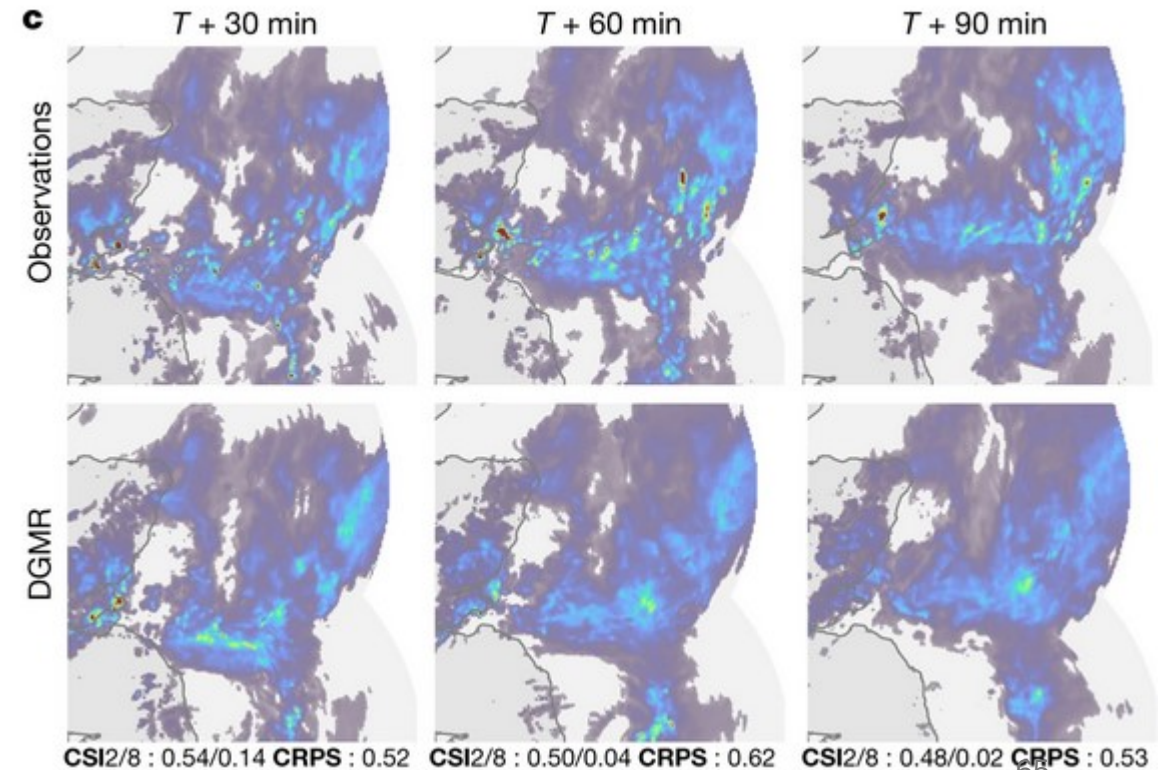


c) Roll out a forecast



Google Graphcast

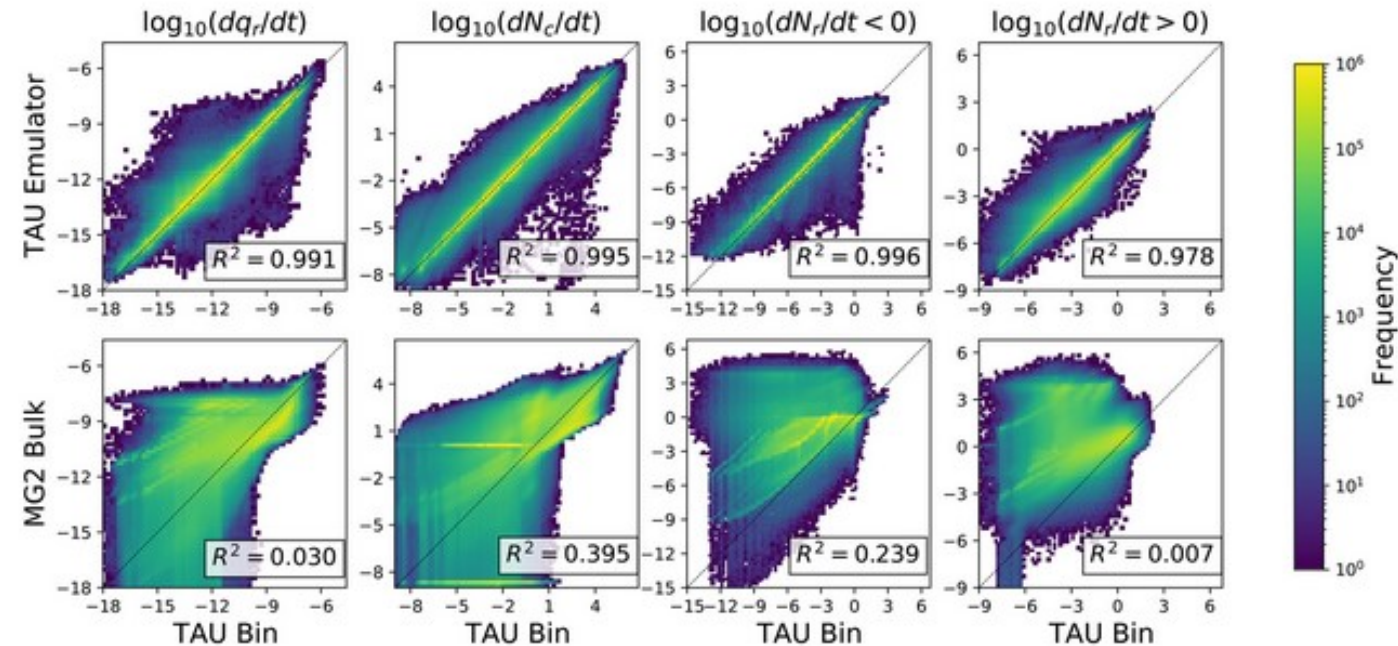
- Lots of matched input/output data
- Easily verifiable
- An ideal ML problem, see GraphCast (Google), Aurora (Microsoft)
- Does it work for climate?



Obs (top) and AI prediction (bottom) for UK rain

Learning parametrisations

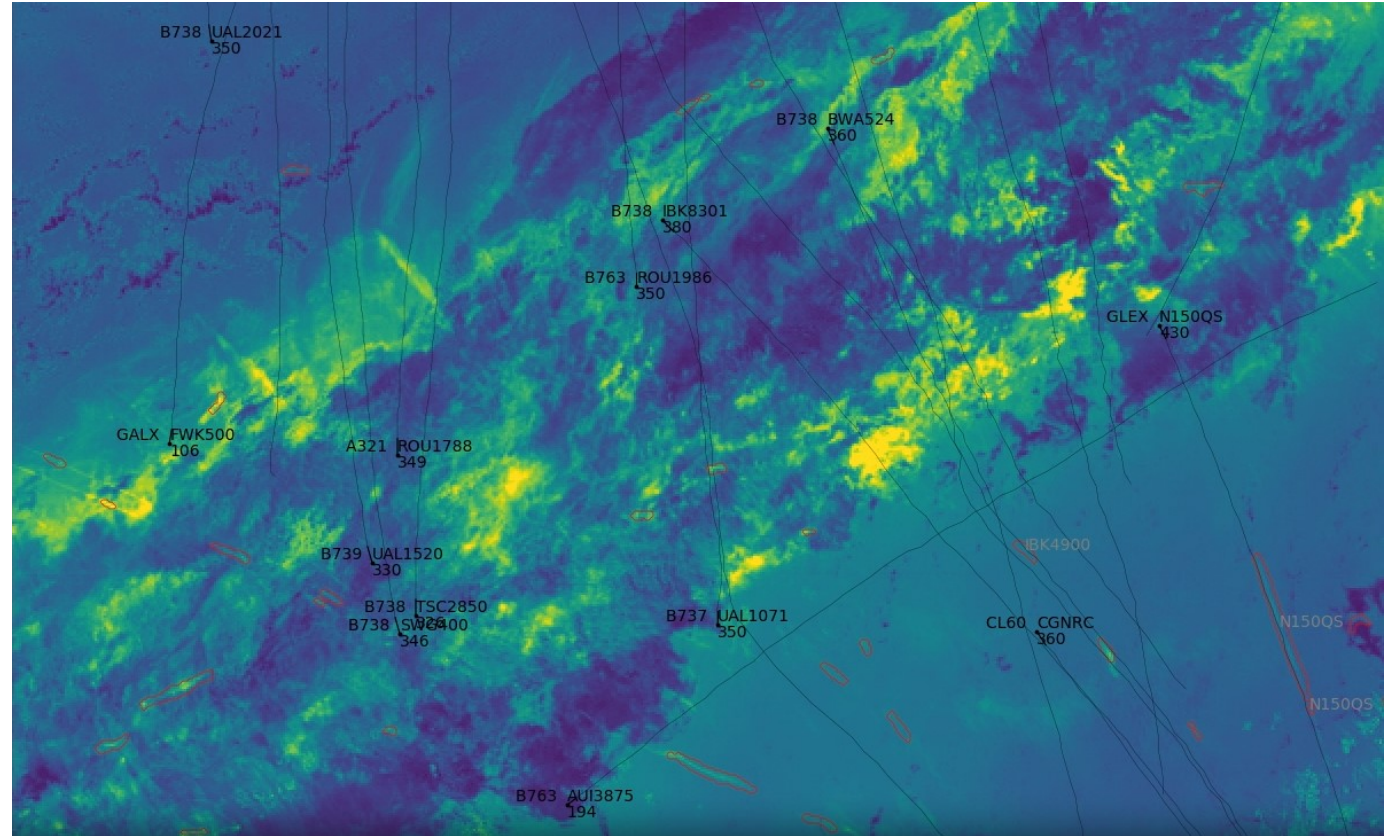
- Models depend on parametrisations of detailed processes
- We can learn these from more complex models
- Could help provide more reliable climate projections
 - Allows model to include more detailed physics



Emulator (top row) and existing parametrisation (bottom) against detailed model (Gettelman et al, (2021))

Identifying features in imagery

- The Earth is big
 - There is a lot of satellite imagery
- ML can help us quickly identify lots of interesting cases
- Here we match contrails (red) to aircraft (black)



Video: <http://www.sp.ph.ic.ac.uk/~erg10/other/aircraft/btd11.mp4>

Method/topic	Resource (Youtube, book, python library, course, etc.)
Machine Learning	https://github.com/josephmisiti/awesome-machine-learning - Curated list of ML resources SciKit-Learn (sklearn), pycaret package gives examples Scipy.optimize “Pattern Recognition and Machine Learning” book
Bayesian Statistics	“Statistical Rethinking” Youtube lecture series and book
Markov Chain Monte Carlo	PyMC, emcee
Gaussian Processes	GPy, GPyTorch, sklearn “Design and Analysis of Computer Experiments” book (WARNING: Mathsy)
Deep Learning	Andrew Ng’s Deep Learning specialisation on Coursera (Can be done for free within time window, otherwise monthly charge)
Neural Networks	Tensorflow, PyTorch, Keras
Differentiable Programming	JAX and associated libraries (diffrax, optax, ...)
Optimisation	Convex Optimisation I and II, Stanford Engineering Everywhere
Software Engineering	Free O’Reilly access: https://www.oreilly.com/library-access/ - <i>Python Cookbook</i> Imperial’s ‘Essential Software Engineering for Researchers’ course (1 credit)

Technique	Pros	Cons
Traditional optimisation (e.g. least-squares with L-M)	Widely used and easy to start Low number of hyperparameters Familiar techniques	Cannot handle noisy data Often require known Jacobian for best performance Uncertainties use Laplace's method Results depend on starting location Scales poorly Can require regularisation
Gradient free optimisation (e.g. Bayesian optimisation)	Finds global minimum No need for gradient information Can handle noisy data	Scales very poorly Choice of kernel (BO) Spend time exploring domain
Differentiable programming	Gradient information for any program Scales well Allows gradients to appear in loss function	Must write code in differentiable form – harder to debug
Splines, etc.	Flexible functions with local extent Data driven predictor	Often want model parameters not predictor function
Markov Chain Monte Carlo	Explore full posterior distribution Proper Bayesian treatment Does not require gradients (apart from HMCMC)	Large number of function evaluations needed
Gaussian processes	Like splines but with uncertainties	Scales poorly
Neural networks	Universal function approximators Scales very well Architectures adapted to problems e.g graphs for molecules	Requires large data set Large number of hyperparameters Model training can be expensive

Additional learning opportunities

- 1-day short course on differentiable simulation
 - Last year: June in Blackett 1004

Course Materials

The course includes lecture material and four Google colab notebooks:

- An introduction to JAX:  [Open in Colab](#)
- Computational graphs and implementing adjoint methods by hand:  [Open in Colab](#)
- Use of differentiable programming libraries to create differentiable simulators:  [Open in Colab](#)
- Summary exercises to test and consolidate your knowledge:  [Open in Colab](#)



[https://github.com/aidancrilly/
MiniCourse-DifferentiableSimulation](https://github.com/aidancrilly/MiniCourse-DifferentiableSimulation)

Links

Please give me some
feedback to improve the
course



Feedback

URL:

<https://forms.gle/37bDBPbWVufSjpB9A>



Code and slides

URL:

https://github.com/aidancrilly/ML_Lecture_Demos

Extras

Ill-conditioned – some maths

- Consider a linear set of equations:

$$\underline{A}\mathbf{x} = \mathbf{b}$$

- Solution can be written in terms of eigenvectors of $\underline{A}$, $\mathbf{v}$

$$\mathbf{x} = \sum_i a_i \mathbf{v}_i \rightarrow a_i = \frac{\mathbf{b} \cdot \mathbf{v}_i}{\lambda_i}$$

- The amplitude of each component is scaled by $1/\lambda_i$
- Therefore, relative sensitivity to perturbations in $\mathbf{b}$

$$\frac{\delta a_{max}}{\delta a_{min}} = \frac{1/\lambda_{min}}{1/\lambda_{max}} = \frac{\lambda_{max}}{\lambda_{min}} \equiv \kappa(\underline{A})$$